M02 — Prácticas integrales: programar y probar APIs REST con .NET 8

Autor: JAIME GALLO

Metodología de trabajo

Cada apartado parte del diseño REST explicado en teoría y culmina en **código ejecutable y peticiones HTTP**. Se trabaja con Visual Studio, ASP.NET Core 8 Minimal API, EF Core 8 y SQL Server Express LocalDB. Los cuatro estados de referencia están en `PROYECTO/2.1` a `PROYECTO/2.4`: no son simulaciones, sino proyectos con endpoints reales probados por HTTP.

La actividad práctica no requiere operaciones Git. El alumnado construye primero la etapa 2.1, incorpora los verbos HTTP en 2.2, conecta la validación en 2.3 y documenta el mismo servicio en 2.4. Para contrastar el resultado, puede abrir por separado los proyectos de referencia. La copia no sustituye la escritura y comprensión de código.

Persistencia: la API del alumno utiliza SQL Server Express LocalDB. Los tests de integración usan EF Core InMemory únicamente para ejecutarse de forma independiente en GitHub Actions. `EnsureCreated` se emplea en desarrollo para simplificar el arranque del ejercicio; no se propone para despliegues productivos.

Cómo construir las cuatro etapas sin duplicar código

Cada punto tiene su **proyecto de referencia terminado** en `PROYECTO/2.1`, `PROYECTO/2.2`, `PROYECTO/2.3` o `PROYECTO/2.4`. La referencia es la evidencia del estado final, no un archivo que deba copiarse antes de empezar. El alumno crea su propia solución, añade cada archivo y modifica lo que ya ha escrito al avanzar de paso. **No se deben pegar dos versiones completas de una misma clase o de un mismo endpoint.**

Los fragmentos se presentan según la etapa en la que el alumno debe introducirlos. El recorrido es **diseño → archivo → código → compilación → petición o prueba → resultado esperado**. Cuando un paso solo examina un comportamiento incorporado anteriormente, se indica expresamente y se evita volver a crear el código. Los retos son ampliaciones opcionales, no requisitos para que los proyectos de referencia compilen.

Criterio de finalización: al completar cada punto, el código de los archivos indicados debe equivaler funcionalmente al proyecto correspondiente y superar sus pruebas. La aplicación local usa SQL Server Express LocalDB; solo los tests sustituyen ese proveedor por InMemory.

Punto 2.1 — Del diseño REST a los primeros endpoints GET

Proyecto comprobado: `PROYECTO/2.1-diseno-apis-rest`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.1.1 — Crear el proyecto ASP.NET Core Empty desde Visual Studio

Resultado de aprendizaje. Crear el proyecto ASP.NET Core Empty desde Visual Studio.

Acción. Archivo > Nuevo > Proyecto > ASP.NET Core Empty. Selecciona C#, .NET 8 y el nombre AceriaApp.Api. No marques autenticación.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Crea la API y comprueba en `AceriaApp.Api.csproj` que `TargetFramework` es `net8.0`. La plantilla vacía debe arrancar antes de introducir EF Core. Si utilizas la carpeta de referencia, no vuelvas a crear el proyecto: ese estado se utiliza para contrastar el resultado.

Por qué funciona así. El programa aún se está preparando: comprobar SDK, arranque y URI antes de introducir persistencia permite aislar errores de herramienta frente a errores de API.

Verificación concreta. Verifica el TargetFramework net8.0 y arranca el proyecto vacío.

Si algo falla. Si falla el inicio, revisa que hayas seleccionado ASP.NET Core Empty y net8.0; no es necesario instalar controladores.

PASO 2.1.2 — Explorar Program.cs, Kestrel y el perfil de arranque

Resultado de aprendizaje. Explorar Program.cs, Kestrel y el perfil de arranque.

Acción. Abre Program.cs y localiza CreateBuilder, Build y Run. Ejecuta con F5, observa la dirección del perfil local y detén Kestrel.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Arranca el proyecto vacío, lee la URL en la salida de Kestrel y detén la ejecución. Este paso no registra endpoints ni necesita LocalDB: una página sin ruta propia no significa que el arranque sea incorrecto.

Por qué funciona así. El programa aún se está preparando: comprobar SDK, arranque y URI antes de introducir persistencia permite aislar errores de herramienta frente a errores de API.

Verificación concreta. Kestrel escucha en el puerto anunciado; no es una prueba de base de datos.

Si algo falla. Si falla el inicio, revisa que hayas seleccionado ASP.NET Core Empty y net8.0; no es necesario instalar controladores.

PASO 2.1.3 — Dibujar los recursos y decidir las dos URIs GET

Resultado de aprendizaje. Dibujar los recursos y decidir las dos URIs GET.

Acción. Diseña las rutas GET /api/v1/ordenes y GET /api/v1/ordenes/{id:guid}. Explica qué recurso identifica cada una.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Escribe las dos rutas en tu cuaderno técnico: colección `/api/v1/ordenes` y miembro `/api/v1/ordenes/{id:guid}`. Distingue que el nombre de ruta no es una tabla SQL y que la restricción `guid` es de enrutamiento.

Por qué funciona así. El programa aún se está preparando: comprobar SDK, arranque y URI antes de introducir persistencia permite aislar errores de herramienta frente a errores de API.

Verificación concreta. Las dos rutas usan sustantivos en plural, y el detalle tiene un Guid.

Si algo falla. Si falla el inicio, revisa que hayas seleccionado ASP.NET Core Empty y net8.0; no es necesario instalar controladores.

PASO 2.1.4 — Crear carpeta Modelo y entidad Orden

Resultado de aprendizaje. Crear carpeta Modelo y entidad Orden.

Acción. Añade la carpeta Modelo. Crea Orden.cs con Guid Id, Cliente, Material, Cantidad, Estado y FechaCreacion; compara con la clase de referencia.

Archivo de trabajo: `src/AceriaApp.Api/Modelo/Orden.cs`.

```
namespace AceriaApp.Api.Modelo;
// INICIO PASO 2.1.4
public sealed class Orden
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Cliente { get; set; } = string.Empty;
    public string Material { get; set; } = string.Empty;
    public int Cantidad { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public DateTime FechaCreacion { get; set; } = DateTime.UtcNow;
}
// FIN PASO 2.1.4
```

Construcción y evidencia del paso. En `Modelo/Orden.cs` incorpora exactamente las seis propiedades del ejemplo y su namespace. Comprueba que `Id` se inicializa con `Guid.NewGuid()`, `Estado` con `Pendiente` y `FechaCreacion` con UTC. Aún no debe haber endpoints.

Por qué funciona así. La entidad y el DTO cumplen funciones diferentes. La primera modela los datos persistidos y el segundo limita el contrato JSON que puede ver el consumidor.

Verificación concreta. La clase Orden compila, y el estado inicial es Pendiente.

Si algo falla. Si no compilan Orden u OrdenDto, comprueba namespaces y tipos. No conviertas la entidad en el contrato JSON.

PASO 2.1.5 — Explicar ID Guid, Estado y FechaCreacion de la entidad

Resultado de aprendizaje. Explicar ID Guid, Estado y FechaCreacion de la entidad.

Acción. En Orden.cs observa que Id se genera por defecto y FechaCreacion emplea UtcNow. Distingue los datos que proporciona el servidor de los del cliente.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. En una instancia `new Orden()`, razona qué valores se generan en el servidor y cuáles vendrán de la petición de un cliente. No copies la misma clase por segunda vez ni cambies el identificador tras persistirla.

Por qué funciona así. La entidad y el DTO cumplen funciones diferentes. La primera modela los datos persistidos y el segundo limita el contrato JSON que puede ver el consumidor.

Verificación concreta. No se reciben el Id ni la fecha desde una petición de creación.

Si algo falla. Si no compilan Orden u OrdenDto, comprueba namespaces y tipos. No conviertas la entidad en el contrato JSON.

PASO 2.1.6 — Crear OrdenDto como contrato de salida

Resultado de aprendizaje. Crear OrdenDto como contrato de salida.

Acción. En Modelo crea OrdenDto.cs como record público con los datos que va a serializar JSON; no lo conviertas en entidad de EF Core.

Archivo de trabajo: **`src/AceriaApp.Api/Modelo/OrdenDto.cs`**.

```
namespace AceriaApp.Api.Modelo;

public sealed record OrdenDto(
    Guid Id, string Cliente, string Material,
    int Cantidad, string Estado, DateTime FechaCreacion);
```

Secuencia de construcción. En `src/AceriaApp.Api/Modelo/OrdenDto.cs` crea primero solo el tipo record. Todavía no tiene método `Desde`; se añadirá en 2.1.7. No copies ahora la versión final.

Por qué funciona así. La entidad y el DTO cumplen funciones diferentes. La primera modela los datos persistidos y el segundo limita el contrato JSON que puede ver el consumidor.

Verificación concreta. El compilador distingue Orden de OrdenDto.

Si algo falla. Si no compilan Orden u OrdenDto, comprueba namespaces y tipos. No conviertas la entidad en el contrato JSON.

PASO 2.1.7 — Programar la conversión OrdenDto.Desde

Resultado de aprendizaje. Programar la conversión OrdenDto.Desde.

Acción. Añade a OrdenDto un método estático Desde(Orden x); construye el record usando las propiedades de una orden.

Archivo de trabajo: **`src/AceriaApp.Api/Modelo/OrdenDto.cs`**.

```
namespace AceriaApp.Api.Modelo;

public sealed record OrdenDto(
    Guid Id, string Cliente, string Material,
    int Cantidad, string Estado, DateTime FechaCreacion)
{
    // PASO 2.1.7: transformar Orden en OrdenDto.
    public static OrdenDto Desde(Orden x) =>
        new(x.Id, x.Cliente, x.Material, x.Cantidad, x.Estado, x.FechaCreacion);
}
```

Secuencia de construcción. En el mismo `OrdenDto.cs`, convierte el record de 2.1.6 en un record con cuerpo: cambia el punto y coma final por llaves y añade dentro `Desde`. El resultado es un único archivo, no dos records.

Por qué funciona así. La entidad y el DTO cumplen funciones diferentes. La primera modela los datos persistidos y el segundo limita el contrato JSON que puede ver el consumidor.

Verificación concreta. El DTO contiene el mismo Id y datos públicos que la entidad.

Si algo falla. Si no compilan Orden u OrdenDto, comprueba namespaces y tipos. No conviertas la entidad en el contrato JSON.

PASO 2.1.8 — Instalar EF Core 8 para SQL Server

Resultado de aprendizaje. Instalar EF Core 8 para SQL Server.

Acción. Desde Administrar paquetes NuGet instala Microsoft.EntityFrameworkCore.SqlServer 8.x en AceriaApp.Api, o utiliza dotnet add package con versión 8.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Instala `Microsoft.EntityFrameworkCore.SqlServer` con una versión 8.x compatible con el SDK y guarda el `.csproj`. Que el paquete se restaure no verifica la conexión; esa verificación llegará tras configurar `AceriaDbContext` y LocalDB.

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. La compilación reconoce UseSqlServer.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.9 — Crear AceriaDbContext y DbSet

Resultado de aprendizaje. Crear AceriaDbContext y DbSet.

Acción. Crea Datos/AceriaDbContext.cs; haz que herede de DbContext y declara DbSet Ordenes.

Archivo de trabajo: **`src/AceriaApp.Api/Datos/AceriaDbContext.cs`**.

```
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;

namespace AceriaApp.Api.Datos;

public sealed class AceriaDbContext(
    DbContextOptions<AceriaDbContext> options) : DbContext(options)
{
    public DbSet<Orden> Ordenes => Set<Orden>();
}
```

Secuencia de construcción. En `src/AceriaApp.Api/Datos/AceriaDbContext.cs` crea únicamente el contexto y el conjunto de órdenes. En 2.1.10 incorporarás la configuración del modelo a esta misma clase.

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. Al compilar, el contexto conoce Orden y puede consultarla.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.10 — Configurar OnModelCreating, clave y longitudes

Resultado de aprendizaje. Configurar OnModelCreating, clave y longitudes.

Acción. En OnModelCreating configura HasKey(Id), las longitudes de Cliente y Material y los campos obligatorios.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Código de este paso. Añade el método al cuerpo del `AceriaDbContext` creado en 2.1.9; no borres el constructor ni `DbSet`. Comprueba que la clase final coincide con la referencia del punto 2.1.

```
// Dentro de AceriaDbContext, antes de su última llave:
protected override void OnModelCreating(ModelBuilder b)
{
```

```
    var e = b.Entity<Orden>();
    e.HasKey(x => x.Id);
    e.Property(x => x.Cliente).HasMaxLength(100).IsRequired();
    e.Property(x => x.Material).HasMaxLength(100).IsRequired();
    e.Property(x => x.Estado).HasMaxLength(24).IsRequired();
}
```

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. El modelo restringe tamaños de columna y reconoce la clave.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.11 — Configurar la cadena de conexión a LocalDB

Resultado de aprendizaje. Configurar la cadena de conexión a LocalDB.

Acción. En appsettings.json añade ConnectionStrings:AceriaApp con (localdb)\MSSQLLocalDB y Database=AceriaAppM02. No sustituyas LocalDB por SQLite.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. En `appsettings.json` debe existir la clave `ConnectionStrings:AceriaApp`. Usa `(localdb)\\MSSQLLocalDB` y la base `AceriaAppM02`; no incluyas credenciales reales en un archivo de prácticas.

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. La cadena se lee desde Configuration y apunta al servidor local.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.12 — Registrar AceriaDbContext mediante AddDbContext

Resultado de aprendizaje. Registrar AceriaDbContext mediante AddDbContext.

Acción. En Program.cs, antes de builder.Build, añade AddDbContext con UseSqlServer y GetConnectionString.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// En Program.cs, entre var builder = WebApplication.CreateBuilder(args)
// y var app = builder.Build():
builder.Services.AddDbContext<AceriaDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("AceriaApp")));
```

Construcción y evidencia del paso. En `Program.cs`, coloca `AddDbContext<AceriaDbContext>` antes de `builder.Build()`. La línea `UseSqlServer` obtiene la cadena `AceriaApp` del archivo anterior. Comprueba que no se crean dos registraciones del contexto.

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. El manejador HTTP podrá recibir AceriaDbContext por inyección.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.13 — Crear la base didáctica solo en Development

Resultado de aprendizaje. Crear la base didáctica solo en Development.

Acción. En Development crea la base de enseñanza con Database.EnsureCreatedAsync usando un scope; deja escrito que en producción se usarían migraciones.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Código de este paso. Este bloque va después de `builder.Build()` y antes de mapear los endpoints. Crea la base de aprendizaje solo en Development; la opción `SkipDatabaseInitialization` se utiliza en los tests de integración.

```
// En Program.cs, justo después de var app = builder.Build():
if (app.Environment.IsDevelopment()
    && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<AceriaDbContext>()
        .Database.EnsureCreatedAsync();
}
```

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. Al arrancar en Windows aparece la base AceriaAppM02 si LocalDB está instalado.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.14 — Escribir el endpoint GET de colección

Resultado de aprendizaje. Escribir el endpoint GET de colección.

Acción. En Program.cs escribe MapGet de /api/v1/ordenes y recibe AceriaDbContext como argumento del lambda.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
app.MapGet("/api/v1/ordenes",
    async (AceriaDbContext db, CancellationToken ct) =>
{
    var ordenes = await db.Ordenes
        .OrderBy(x => x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes");
```

Secuencia de construcción. Escribe en `Program.cs` el GET funcional con persistencia y conversión a DTO. En el siguiente paso se añade `AsNoTracking`; no debes duplicar la ruta.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. El endpoint queda registrado y responde a GET.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.15 — Utilizar AsNoTracking y proyectar a DTO

Resultado de aprendizaje. Utilizar AsNoTracking y proyectar a DTO.

Acción. Dentro de MapGet consulta db.Ordenes.AsNoTracking, materializa la lista y aplica OrdenDto.Desde a cada entidad.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Código de este paso. Modifica la consulta existente y verifica que la ruta, el tipo de respuesta y la ordenación siguen siendo los mismos. Esta consulta es la versión final del endpoint de colección.

```
// En el MapGet de 2.1.14, sustituye solo la consulta:
var ordenes = await db.Ordenes.AsNoTracking()
    .OrderBy(x => x.FechaCreacion).ToListAsync(ct);
// Mantén la llamada: return Results.Ok(ordenes.Select(OrdenDto.Desde));
```

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. La salida expone DTOs y no instancias de la entidad EF Core.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.16 — Ejecutar GET y verificar 200 con lista vacía

Resultado de aprendizaje. Ejecutar GET y verificar 200 con lista vacía.

Acción. Abre AceriaApp.http, ejecuta GET de colección y comprueba el código y el JSON aunque la tabla esté vacía.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Con LocalDB disponible ejecuta el GET de colección desde `AceriaApp.http`: espera **HTTP 200** y un array JSON (`[]` si aún no hay órdenes). Un array vacío no es un 404.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. La respuesta es 200 y un arreglo JSON; no 404.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.17 — Escribir el GET individual con

Resultado de aprendizaje. Escribir el GET individual con {id:guid}.

Acción. Añade MapGet /api/v1/ordenes/{id:guid} y recibe Guid id y DbContext inyectado.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
app.MapGet("/api/v1/ordenes/{id:guid}",
    async (Guid id, AceriaDbContext db) =>
```

```
{
    var orden = await db.Ordenes.AsNoTracking()
        .FirstOrDefaultAsync(x => x.Id == id);
    IResult response = orden is null
        ? Results.NotFound()
        : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes");
```

Secuencia de construcción. Añade **una sola vez** el GET de detalle en `Program.cs`. El paso 2.1.18 analizará la rama `NotFound`, ya presente aquí; no añadirá un segundo endpoint.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. La ruta coincide con un identificador Guid válido.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.18 — Devolver NotFound cuando no existe la orden

Resultado de aprendizaje. Devolver NotFound cuando no existe la orden.

Acción. Busca mediante FirstOrDefaultAsync y devuelve Results.NotFound cuando la orden no existe; usa Results.Ok cuando aparece.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Reutiliza el GET de detalle de 2.1.17 y solicita un `Guid` inexistente pero bien formado. Debes obtener **HTTP 404**; esa ruta llega al manejador y este constata que no existe una fila.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. Un GUID nuevo devuelve 404 sin excepción 500.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.19 — Comprobar diferencias entre ruta incorrecta y recurso ausente

Resultado de aprendizaje. Comprobar diferencias entre ruta incorrecta y recurso ausente.

Acción. Envía una ruta con identificador que no es GUID y otra con GUID inexistente; anota qué parte resuelve el enrutador y cuál el manejador.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Compara una ruta con `id=no-es-guid` y otra con `id` GUID válido pero inexistente. Ambos casos pueden devolver 404, pero el primero lo decide el enrutamiento y el segundo la consulta del manejador. Utiliza puntos de interrupción para distinguirlos.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. Comprendes la diferencia entre plantilla que no coincide y recurso ausente.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.20 — Crear AceriaApp.http y enviar peticiones desde Visual Studio

Resultado de aprendizaje. Crear AceriaApp.http y enviar peticiones desde Visual Studio.

Acción. En Visual Studio abre o crea AceriaApp.http, define @Host y dos peticiones GET con la ruta versionada.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Crea `AceriaApp.http` con una variable para el host de ejecución y dos solicitudes GET separadas por ###. No escribas puertos fijos si el perfil local utiliza uno distinto: toma la URL de la salida de la API.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. Puedes ejecutar ambos GET sin copiar URLs del navegador.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

Caso práctico completo — Del navegador hasta una consulta EF Core

Este caso demuestra el trayecto completo de una petición GET antes de añadir pruebas. Abre `src/AceriaApp.Api/Program.cs` y coloca un punto de interrupción justo dentro de `app.MapGet("/api/v1/ordenes", ...)`. Ejecuta la API con F5 usando el perfil Development y abre `AceriaApp.http`. Envía `GET /api/v1/ordenes`. Cuando Visual Studio se detenga, inspecciona el argumento `AceriaDbContext db`. No lo ha creado el cliente HTTP: lo resolvió el contenedor de dependencias a partir de `AddDbContext`, configurado antes de construir la aplicación.

Continúa paso por paso. `db.Ordenes` representa la tabla de órdenes; `AsNoTracking()` indica que EF Core no va a mantener seguimiento de cambios en las entidades recuperadas. Esta opción resulta apropiada cuando no se pretende modificar esas mismas entidades. `OrderBy(x => x.FechaCreacion)` establece un orden reproducible de presentación, y `ToListAsync` ejecuta la consulta. Si no hay filas, el resultado será una lista vacía, que debe traducirse en un estado 200 con `[]` en el cuerpo JSON.

La llamada `OrdenDto.Desde` realiza el límite entre persistencia y representación. El cliente no recibe una instancia del contexto ni detalles sobre una tabla; recibe un objeto JSON con los campos públicos de la orden. La ruta de detalle añade una comprobación: si el Guid es válido pero la consulta no encuentra ninguna fila, el servidor responde 404. Comprueba esta situación con el identificador inexistente del archivo HTTP. Si recibes una excepción de conexión en lugar de 200 o 404, revisa que LocalDB está instalado y que la cadena de conexión apunta a `(localdb)\MSSQLLocalDB`.

Demostración de aprendizaje: explica por qué `Results.Ok` y `Results.NotFound` pertenecen al contrato HTTP y no al modelo de dominio. Cambia temporalmente una ruta en Program.cs y observa cómo deja de coincidir la petición anterior; restaura el código y vuelve a comprobarlo.

PASO 2.1.21 — Crear el proyecto xUnit de pruebas HTTP

Resultado de aprendizaje. Crear el proyecto xUnit de pruebas HTTP.

Acción. Crea un proyecto xUnit AceriaApp.Api.Tests y referencia AceriaApp.Api; instala Microsoft.AspNetCore.Mvc.Testing 8.

Archivo de trabajo: **`tests/AceriaApp.Api.Tests/ApiFactory.cs`**.

```
# Desde la carpeta raíz del punto 2.1:
dotnet new xunit -n AceriaApp.Api.Tests -o tests/AceriaApp.Api.Tests --framework net8.0
dotnet add tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj reference src/AceriaApp.Api/
AceriaApp.Api.csproj
dotnet add tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj package
Microsoft.AspNetCore.Mvc.Testing --version 8.0.21
dotnet add tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj package
Microsoft.EntityFrameworkCore.InMemory --version 8.0.21
```

Secuencia de construcción. Este paso crea el proyecto de tests y sus referencias. Si ya existe por haber abierto el estado de referencia, examina el `.csproj` sin crear otro proyecto. El archivo `ApiFactory.cs` se crea en el **paso 2.1.22**, no aquí.

Por qué funciona así. WebApplicationFactory prueba la aplicación como servidor HTTP: permite distinguir compilación C# de comportamiento observable del enrutamiento y la serialización.

Verificación concreta. Las clases de tests referencian Program de la API.

Si algo falla. Si falla ApiFactory, comprueba Program partial y el registro de DbContext con InMemory solo para tests.

PASO 2.1.22 — Configurar WebApplicationFactory y base aislada para tests

Resultado de aprendizaje. Configurar WebApplicationFactory y base aislada para tests.

Acción. Crea ApiFactory derivada de WebApplicationFactory; sustituye solo para pruebas el proveedor por EF Core InMemory.

Archivo de trabajo: **`tests/AceriaApp.Api.Tests/ApiFactory.cs`**.

```
using AceriaApp.Api.Datos;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.TestHost;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.1.22: HTTP real; base en memoria SOLO en tests automatizados.
public sealed class ApiFactory:WebApplicationFactory<Program>
{
    private readonly string _databaseName="Tests-"+Guid.NewGuid();
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Development");
        builder.UseSetting("SkipDatabaseInitialization","true");
        builder.ConfigureTestServices(services=>
        {
            services.RemoveAll<DbContextOptions<AceriaDbContext>>();
            services.AddDbContext<AceriaDbContext>(opt=>
                opt.UseInMemoryDatabase(_databaseName));
        });
    }
}
```

Construcción y evidencia del paso. Ahora sí crea `tests/AceriaApp.Api.Tests/ApiFactory.cs`. Debe heredar de `WebApplicationFactory<Program>`, establecer `SkipDatabaseInitialization=true` y reemplazar el proveedor del contexto **solo dentro del host de pruebas**. Contrasta el archivo con la referencia de 2.1.

Por qué funciona así. WebApplicationFactory prueba la aplicación como servidor HTTP: permite distinguir compilación C# de comportamiento observable del enrutamiento y la serialización.

Verificación concreta. Las pruebas arrancan ASP.NET Core en memoria; la aplicación real conserva LocalDB.

Si algo falla. Si falla ApiFactory, comprueba Program partial y el registro de DbContext con InMemory solo para tests.

PASO 2.1.23 — Probar 200, 404 y DTO mediante GetTests

Resultado de aprendizaje. Probar 200, 404 y DTO mediante GetTests.

Acción. Crea GetTests y tres casos: colección 200, GUID inexistente 404, orden insertada recuperada como OrdenDto.

Archivo de trabajo: **`tests/AceriaApp.Api.Tests/GetTests.cs`**.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.Extensions.DependencyInjection;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.1.23: dos peticiones HTTP, código y DTO.
public class GetTests
{
    [Fact] public async Task ListaVacia_Es200()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes");
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
    }
    [Fact] public async Task RecursoInexistente_Es404()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes/"+Guid.NewGuid());
        Assert.Equal(HttpStatusCode.NotFound,r.StatusCode);
    }
    [Fact] public async Task RecursoExistente_ExponeDto()
    {
        using var f=new ApiFactory();
        var id=Guid.NewGuid();
        using(var scope=f.Services.CreateScope())
        {
            var db=scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
            db.Ordenes.Add(new Orden{Id=id,Cliente="ACX-001",Material="CHAPA",Cantidad=10});
            await db.SaveChangesAsync();
        }
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes/"+id);
        var dto=await r.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
        Assert.Equal(id,dto?.Id);
    }
}
```

Construcción y evidencia del paso. Crea `GetTests.cs` y ejecuta `dotnet test tests/AceriaApp.Api.Tests`. Se esperan tres casos: colección 200, GUID inexistente 404 y DTO recuperado después de insertar la entidad en la base aislada de la factoría.

Por qué funciona así. WebApplicationFactory prueba la aplicación como servidor HTTP: permite distinguir compilación C# de comportamiento observable del enrutamiento y la serialización.

Verificación concreta. dotnet test ejecuta los tres casos con resultado satisfactorio.

Si algo falla. Si falla ApiFactory, comprueba Program partial y el registro de DbContext con InMemory solo para tests.

PASO 2.1.24 — Reto: incorporar consulta por cliente y explicar su URI

Resultado de aprendizaje. Reto: incorporar consulta por cliente y explicar su URI.

Acción. Como reto, acepta ?cliente=ACX-001 en el GET de colección y aplica Where solo cuando se recibe cliente; crea un test.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. El filtrado por `cliente` es un **reto opcional**, no un endpoint presente en el código base de 2.1. Añádelo en tu propia copia, crea su test y documenta la diferencia entre colección filtrada vacía y un recurso individual ausente.

Por qué funciona así. El filtrado de la colección conserva la idea de recurso y añade una consulta con parámetros, sin transformar el endpoint en una operación RPC.

Verificación concreta. La colección devuelve únicamente órdenes del cliente indicado.

Si algo falla. Si el filtro no cambia el resultado, revisa el nombre cliente en la query y la condición Where.

Caso práctico completo — Probar una API de lectura sin depender de una base instalada en CI

Abre el proyecto de pruebas y localiza `ApiFactory.cs`. Esta clase deriva de `WebApplicationFactory<Program>` y arranca una aplicación real de ASP.NET Core en memoria. La aplicación del alumno sigue conectándose a LocalDB cuando se ejecuta en Windows; exclusivamente dentro de la fábrica de tests se sustituye el proveedor por EF Core InMemory. Esa decisión mantiene las pruebas reproducibles y permite que se ejecuten en un runner Linux, pero no autoriza a utilizar InMemory como sustituto de SQL Server para verificar todo su comportamiento.

El primer test consulta la colección vacía y espera 200; el segundo construye un Guid nuevo, lo incorpora a la ruta de detalle y espera 404. Para comprobar el caso positivo se introduce una entidad Orden en la base de la prueba, se confirma SaveChangesAsync y después se envía un GET al mismo identificador. La respuesta se deserializa a OrdenDto. El test debe contrastar al menos el identificador y uno de los campos, porque comprobar solamente un 200 no garantiza que haya aparecido la orden correcta.

Ejecuta desde la terminal integrada `dotnet test` indicando el proyecto `tests/AceriaApp.Api.Tests`. Si un test que insertó una entidad recibe 404, revisa que el mismo contexto de la fábrica comparta base y nombre durante la inserción y el GET. Si el código compila pero el servidor de pruebas no arranca, comprueba que el programa expone `public partial class Program` y que el proyecto de tests referencia la API.

Reto de cierre del punto: escribe una prueba para el filtro opcional por cliente de la colección. Inserta dos órdenes de clientes diferentes, haz la consulta con un parámetro `cliente` y compara los identificadores. La prueba debe rechazar una implementación que devuelva todas las filas.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`GET /api/v1/ordenes`	200 + JSON
`GET /api/v1/ordenes/{id}`	200 / 404

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el

código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.2 — Recursos, verbos, persistencia y respuestas HTTP

Proyecto comprobado: `PR0YECT0/2.2-recursos-verbos-codigos`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.2.1 — Recuperar el estado GET funcional sin modificarlo

Resultado de aprendizaje. Recuperar el estado GET funcional sin modificarlo.

Acción. Abre la etapa 2.1 ya programada; copia tu solución de trabajo a la carpeta del punto 2.2 y ejecuta los GET existentes.

Referencia para comprobar el resultado: los archivos del proyecto `PR0YECT0/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Crea la copia de trabajo para 2.2 a partir de tu solución final 2.1; verifica primero que sus dos GET siguen dando 200 y 404. El proyecto de referencia de 2.2 es el estado **final** de esta etapa, no el punto de partida obligatorio.

Por qué funciona así. El diseño de las rutas y los contratos evita recibir propiedades de servidor desde JSON. El resultado público se devuelve mediante OrdenDto, no directamente desde la entidad.

Verificación concreta. La lista sigue devolviendo 200 y el detalle inexistente 404.

Si algo falla. Si el JSON puede cambiar el ID o Estado, el contrato de entrada está exponiendo datos internos; utiliza CrearOrdenRequest.

PASO 2.2.2 — Diseñar la tabla verbo, URI, petición y respuesta

Resultado de aprendizaje. Diseñar la tabla verbo, URI, petición y respuesta.

Acción. Construye una tabla con GET/POST/PUT/PATCH/DELETE y las URIs de órdenes; anota cuándo será 201, 204, 400, 404, 409 y 422.

Referencia para comprobar el resultado: los archivos del proyecto `PR0YECT0/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Prepara una tabla de operaciones de **las rutas que vas a implementar**, no de hipotéticos recursos adicionales. Anota `Location` para POST y el efecto sobre datos persistidos para PUT, PATCH y DELETE.

Por qué funciona así. El diseño de las rutas y los contratos evita recibir propiedades de servidor desde JSON. El resultado público se devuelve mediante OrdenDto, no directamente desde la entidad.

Verificación concreta. La tabla coincide con las rutas que programarás, sin acciones Git.

Si algo falla. Si el JSON puede cambiar el ID o Estado, el contrato de entrada está exponiendo datos internos; utiliza CrearOrdenRequest.

PASO 2.2.3 — Separar contrato de entrada y DTO de salida

Resultado de aprendizaje. Separar contrato de entrada y DTO de salida.

Acción. En la carpeta Contratos dibuja qué acepta el cliente y qué genera el servidor. No recibas Orden directamente desde JSON.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. En `Contratos/ContratosOrden.cs` separa lo que escribe el consumidor (`CrearOrdenRequest`, `ActualizarCantidadRequest`) de `OrdenDto`, que contiene datos generados por el servidor. El cliente no debe asignar `Id` ni `FechaCreacion`.

Por qué funciona así. El diseño de las rutas y los contratos evita recibir propiedades de servidor desde JSON. El resultado público se devuelve mediante OrdenDto, no directamente desde la entidad.

Verificación concreta. Id, Estado y FechaCreacion quedan fuera del contrato de creación.

Si algo falla. Si el JSON puede cambiar el ID o Estado, el contrato de entrada está exponiendo datos internos; utiliza CrearOrdenRequest.

PASO 2.2.4 — Crear CrearOrdenRequest y ActualizarCantidadRequest

Resultado de aprendizaje. Crear CrearOrdenRequest y ActualizarCantidadRequest.

Acción. Crea ContratosOrden.cs con CrearOrdenRequest(Cliente, Material, Cantidad) y ActualizarCantidadRequest(Cantidad).

Archivo de trabajo: **`src/AceriaApp.Api/Contratos/ContratosOrden.cs`**.

```
namespace AceriaApp.Api.Contratos;
// INICIO PASO 2.2.4
public sealed record CrearOrdenRequest(string Cliente,string Material,int Cantidad);
public sealed record ActualizarCantidadRequest(int Cantidad);
// FIN PASO 2.2.4
```

Construcción y evidencia del paso. Crea dos records públicos en `ContratosOrden.cs` y añade el namespace `AceriaApp.Api.Contratos`. Comprueba que el proyecto puede referenciarlos desde `Program.cs` sin introducir todavía FluentValidation.

Por qué funciona así. El diseño de las rutas y los contratos evita recibir propiedades de servidor desde JSON. El resultado público se devuelve mediante OrdenDto, no directamente desde la entidad.

Verificación concreta. Los dos records compilan y se distinguen del OrdenDto de salida.

Si algo falla. Si el JSON puede cambiar el ID o Estado, el contrato de entrada está exponiendo datos internos; utiliza CrearOrdenRequest.

PASO 2.2.5 — Programar MapPost y recibir JSON de una orden

Resultado de aprendizaje. Programar MapPost y recibir JSON de una orden.

Acción. En Program.cs declara MapPost /api/v1/ordenes con CrearOrdenRequest y AceriaDbContext. Explica cómo se enlaza JSON.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
```

```
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Construcción y evidencia del paso. En `Program.cs` agrega **un solo MapPost**. La versión completa se muestra en este paso para que compile. Los pasos 2.2.6 y 2.2.7 explican y comprueban las sentencias de persistencia y `Location` que ya aparecen en ese manejador.

Por qué funciona así. El POST crea un recurso y debe confirmar la inserción mediante SaveChangesAsync. CreatedAtRoute devuelve 201 y un enlace Location utilizable mediante GET.

Verificación concreta. La URL acepta POST con Content-Type application/json.

Si algo falla. Si el POST no persiste, verifica SaveChangesAsync; si Location falta, revisa CreatedAtRoute y el nombre de GET.

PASO 2.2.6 — Persistir la orden con Add y SaveChangesAsync

Resultado de aprendizaje. Persistir la orden con Add y SaveChangesAsync.

Acción. Dentro de POST crea una entidad Orden con los campos del contrato, ejecuta db.Ordenes.Add y SaveChangesAsync.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Construcción y evidencia del paso. Localiza `db.Ordenes.Add(orden)` y `SaveChangesAsync()` en el POST creado. Comprueba que sin `SaveChangesAsync` una orden nueva no se mantiene tras otra petición. No añadas un segundo POST.

Por qué funciona así. El POST crea un recurso y debe confirmar la inserción mediante SaveChangesAsync. CreatedAtRoute devuelve 201 y un enlace Location utilizable mediante GET.

Verificación concreta. Una nueva fila queda persistida en LocalDB.

Si algo falla. Si el POST no persiste, verifica SaveChangesAsync; si Location falta, revisa CreatedAtRoute y el nombre de GET.

PASO 2.2.7 — Devolver 201 y Location con CreatedAtRoute

Resultado de aprendizaje. Devolver 201 y Location con CreatedAtRoute.

Acción. Devuelve Results.CreatedAtRoute usando el nombre ObtenerOrdenPorId y el identificador generado.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// Este retorno ya está en el único MapPost creado en 2.2.5.
// PASO 2.2.7: comprueba que el GET tiene el nombre ObtenerOrdenPorId.
return Results.CreatedAtRoute(
    "ObtenerOrdenPorId",
    new { id = orden.Id },
    OrdenDto.Desde(orden));
```

Construcción y evidencia del paso. La sentencia `CreatedAtRoute("ObtenerOrdenPorId", ...)` del **mismo** POST debe generar 201 y la cabecera `Location`; revisa que el nombre coincida con el GET de detalle construido en 2.1.

Por qué funciona así. El POST crea un recurso y debe confirmar la inserción mediante SaveChangesAsync. CreatedAtRoute devuelve 201 y un enlace Location utilizable mediante GET.

Verificación concreta. El resultado es 201 con cabecera Location.

Si algo falla. Si el POST no persiste, verifica SaveChangesAsync; si Location falta, revisa CreatedAtRoute y el nombre de GET.

PASO 2.2.8 — Enviar POST y consultar por GET la orden creada

Resultado de aprendizaje. Enviar POST y consultar por GET la orden creada.

Acción. Envía POST con ACX-001, CHAPA-3MM y 10; copia el identificador y ejecuta GET de detalle.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Construcción y evidencia del paso. Envía POST con `cliente=ACX-001`, `material=CHAPA-3MM` y `cantidad=10`. Copia el `Guid` retornado y ejecuta GET al recurso; comprueba que coincide con `Location` y con los campos persistidos.

Por qué funciona así. El POST crea un recurso y debe confirmar la inserción mediante SaveChangesAsync. CreatedAtRoute devuelve 201 y un enlace Location utilizable mediante GET.

Verificación concreta. GET muestra los mismos valores guardados por POST.

Si algo falla. Si el POST no persiste, verifica SaveChangesAsync; si Location falta, revisa CreatedAtRoute y el nombre de GET.

PASO 2.2.9 — Rechazar provisionalmente peticiones sin datos

Resultado de aprendizaje. Rechazar provisionalmente peticiones sin datos.

Acción. En 2.2, antes de introducir FluentValidation, añade un BadRequest provisional ante Cliente o Material vacíos y Cantidad no positiva.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. La comprobación provisional de campos obligatorios vive en el POST de esta etapa. Prueba `cliente` vacío y `cantidad=0`: espera 400. Cuando llegues a 2.3 se reemplaza este mecanismo por los filtros, sin duplicar validaciones.

Por qué funciona así. Un cuerpo incorrecto es distinto de una regla del negocio. En 2.2 el 400 es provisional; 501 unidades reciben 422 por la capacidad limitada del taller.

Verificación concreta. La petición inválida produce 400 y no crea una orden.

Si algo falla. Si un 501 da 400, la condición provisional está mezclada con la política de capacidad; distingue ambos casos.

PASO 2.2.10 — Comprobar la política del taller y respuesta 422

Resultado de aprendizaje. Comprobar la política del taller y respuesta 422.

Acción. Rechaza Cantidad superior a 500 por política de capacidad con Results.Problem statusCode 422.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Con `cantidad=501` y los otros campos válidos, el manejador alcanza la política de capacidad y devuelve 422; no confundas este límite de negocio con la validación semántica de `cantidad<=0`.

Por qué funciona así. Un cuerpo incorrecto es distinto de una regla del negocio. En 2.2 el 400 es provisional; 501 unidades reciben 422 por la capacidad limitada del taller.

Verificación concreta. El JSON de capacidad 501 produce 422; se distingue de 400.

Si algo falla. Si un 501 da 400, la condición provisional está mezclada con la política de capacidad; distingue ambos casos.

PASO 2.2.11 — Programar MapPut con identificador Guid

Resultado de aprendizaje. Programar MapPut con identificador Guid.

Acción. Declara MapPut /api/v1/ordenes/{id:guid} con Guid, CrearOrdenRequest y AceriaDbContext.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.11: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")


;
```

Construcción y evidencia del paso. Crea un único `MapPut("/api/v1/ordenes/{id:guid}", ...);` el fragmento incluye las ramas necesarias para compilar. El paso 2.2.12 estudiará y probará el reemplazo de los tres campos editables de ese mismo endpoint.

Por qué funciona así. PUT sustituye los campos editables de una orden existente. Si el Guid no identifica un recurso debe devolver 404, no crear silenciosamente otro.

Verificación concreta. PUT puede buscar un recurso por identificador.

Si algo falla. Si el PUT no actualiza, revisa FindAsync(id) y SaveChangesAsync. El GUID no debe cambiar.

PASO 2.2.12 — Modificar Cliente, Material y Cantidad con PUT

Resultado de aprendizaje. Modificar Cliente, Material y Cantidad con PUT.

Acción. En el PUT carga FindAsync(id), devuelve 404 si no existe y sustituye los tres campos editables; SaveChangesAsync.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Construcción y evidencia del paso. En el `MapPut` anterior verifica `FindAsync(id)`, 404 para recurso ausente, 409 para orden finalizada y 422 para capacidad. Solo si es editable cambia `Cliente`, `Material` y `Cantidad`, y guarda los cambios.

Por qué funciona así. PUT sustituye los campos editables de una orden existente. Si el Guid no identifica un recurso debe devolver 404, no crear silenciosamente otro.

Verificación concreta. PUT válido responde 204 y mantiene el mismo Guid.

Si algo falla. Si el PUT no actualiza, revisa FindAsync(id) y SaveChangesAsync. El GUID no debe cambiar.

Caso práctico completo — Una transacción observable: POST, Location, GET y PUT

El primer recorrido de escritura empieza con el método POST. Ejecuta la aplicación y abre `AceriaApp.http`. Envía un cuerpo JSON válido:

```
{"cliente":"ACX-001","material":"CHAPA-3MM","cantidad":10}
```

Utiliza la URI `/api/v1/ordenes` y el verbo POST. La operación construye una entidad nueva de Orden con los datos del contrato. El identificador y la fecha se asignan en el servidor. Cuando `SaveChangesAsync` confirma la inserción, `Results.CreatedAtRoute` prepara un estado 201 con una cabecera `Location`. Abre esa cabecera: no es un mensaje decorativo, sino una ruta que puede usarse después para recuperar la orden.

Envía ahora GET a la URI de `Location` y comprueba que el JSON contiene cliente ACX-001, material CHAPA-3MM y cantidad 10. Guarda el Guid obtenido: lo necesitarás para PUT, PATCH y DELETE. Si GET no encuentra la orden, pregunta qué base de datos usa cada petición y si el POST realizó `SaveChangesAsync`. El hecho de que el POST devuelva un cuerpo con un ID no demuestra por sí solo que se haya insertado una fila.

Para entender PUT, envía a `/api/v1/ordenes/{id}` un cuerpo con cliente ACX-002, material CHAPA-5MM y cantidad 14. El endpoint localiza la orden mediante `FindAsync(id)`, modifica tres propiedades editables y persiste el cambio. El resultado normal es 204, sin un cuerpo de respuesta que copiar. Ejecuta un nuevo GET: el Guid debe ser el mismo, mientras que los tres campos editables reflejan los nuevos valores.

Comprobación: compara lo que cambió y lo que permaneció intacto. El ID, el Estado y la FechaCreacion son valores controlados por el servidor. Si un PUT crea silenciosamente una orden con un Guid inexistente, no está cumpliendo el contrato concreto de este ejemplo.

PASO 2.2.13 — Comprobar 204 y consultar por GET los cambios

Resultado de aprendizaje. Comprobar 204 y consultar por GET los cambios.

Acción. Tras PUT ejecuta GET del mismo recurso y comprueba Cliente, Material y Cantidad nuevos.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Construcción y evidencia del paso. Prueba PUT con el GUID creado y campos diferentes: espera **204 sin cuerpo**. Ejecuta GET después para observar los valores nuevos, porque el 204 por sí solo no demuestra qué se almacenó.

Por qué funciona así. PUT sustituye los campos editables de una orden existente. Si el Guid no identifica un recurso debe devolver 404, no crear silenciosamente otro.

Verificación concreta. La consulta evidencia que el cambio se persistió.

Si algo falla. Si el PUT no actualiza, revisa FindAsync(id) y SaveChangesAsync. El GUID no debe cambiar.

PASO 2.2.14 — Programar MapPatch para cambiar solo Cantidad

Resultado de aprendizaje. Programar MapPatch para cambiar solo Cantidad.

Acción. Declara MapPatch /api/v1/ordenes/{id:guid}/cantidad con ActualizarCantidadRequest.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.14: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad<=0) return Results.BadRequest(new {error="Cantidad debe ser positiva."});
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")


;
```

Construcción y evidencia del paso. Crea el único `MapPatch` especializado en `/cantidad`. No se está implementando JSON Patch genérico: `ActualizarCantidadRequest` contiene una sola propiedad. Comprueba que el handler conserva los demás campos.

Por qué funciona así. PATCH realiza un cambio parcial de Cantidad y conserva los demás datos. Su resultado debe verificarse con una lectura posterior.

Verificación concreta. PATCH admite un cuerpo que contiene solo Cantidad.

Si algo falla. Si PATCH cambia otros campos, no reasignes Cliente ni Material desde ActualizarCantidadRequest.

PASO 2.2.15 — Comprobar que PATCH no modifica los demás campos

Resultado de aprendizaje. Comprobar que PATCH no modifica los demás campos.

Acción. Guarda la nueva cantidad y devuelve Results.Ok(OrdenDto.Desde(orden)); consulta después y verifica que Cliente sigue igual.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Construcción y evidencia del paso. Desde el endpoint PATCH anterior cambia `Cantidad` y comprueba su 200 y DTO. A continuación haz GET y compara `Cliente` y `Material` anteriores; no crees otro handler para guardar.

Por qué funciona así. PATCH realiza un cambio parcial de Cantidad y conserva los demás datos. Su resultado debe verificarse con una lectura posterior.

Verificación concreta. Se obtiene 200 y no cambian datos no incluidos en el parche.

Si algo falla. Si PATCH cambia otros campos, no reasignes Cliente ni Material desde ActualizarCantidadRequest.

PASO 2.2.16 — Programar MapDelete para eliminar una orden

Resultado de aprendizaje. Programar MapDelete para eliminar una orden.

Acción. Declara MapDelete /api/v1/ordenes/{id:guid}; usa FindAsync, Remove y SaveChangesAsync.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.16: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")

;
```

Construcción y evidencia del paso. Registra `MapDelete` con su consulta `FindAsync`, bloqueo 409 por estado, `Remove`, `SaveChangesAsync` y resultado 204. No confundas eliminación lógica con borrado físico: este ejercicio utiliza `Remove`.

Por qué funciona así. DELETE quita un recurso persistido. La consulta posterior puede devolver 404; esa diferencia no impide que la eliminación sea idempotente.

Verificación concreta. DELETE existente devuelve 204.

Si algo falla. Si DELETE devuelve éxito pero GET encuentra la orden, revisa Remove y SaveChangesAsync.

PASO 2.2.17 — Verificar 204 seguido de GET 404

Resultado de aprendizaje. Verificar 204 seguido de GET 404.

Acción. Llama GET al recurso recién eliminado y después repite DELETE sobre su Id.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Construcción y evidencia del paso. Al borrar una orden `Pendiente`, espera 204. Un GET posterior y un segundo DELETE deben indicar que ya no existe. La idempotencia se razona por el estado final, no por repetir el mismo código HTTP.

Por qué funciona así. DELETE quita un recurso persistido. La consulta posterior puede devolver 404; esa diferencia no impide que la eliminación sea idempotente.

Verificación concreta. El GET devuelve 404 y el recurso sigue ausente.

Si algo falla. Si DELETE devuelve éxito pero GET encuentra la orden, revisa Remove y SaveChangesAsync.

PASO 2.2.18 — Programar la transición de estado finalizar

Resultado de aprendizaje. Programar la transición de estado finalizar.

Acción. Declara POST /api/v1/ordenes/{id:guid}/finalizar para establecer Estado=Finalizada y guardar.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.18: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
```

```
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")

;

app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Construcción y evidencia del paso. Crea el endpoint POST `/api/v1/ordenes/{id:guid}/finalizar` como una transición explícita. Solo una orden existente y todavía pendiente puede finalizarse; repetir la transición devuelve 409.

Por qué funciona así. La transición Finalizada cambia el estado de negocio. PUT, PATCH y DELETE deben respetarlo y producir 409 si se intenta una operación incompatible.

Verificación concreta. La transición devuelve 200 y el DTO indica Finalizada.

Si algo falla. Si una orden finalizada cambia de estado o desaparece, comprueba la condición Estado antes de escribir.

PASO 2.2.19 — Bloquear PUT y PATCH para órdenes finalizadas

Resultado de aprendizaje. Bloquear PUT y PATCH para órdenes finalizadas.

Acción. En PUT y PATCH comprueba Estado=Finalizada antes de modificar y devuelve Problem con 409.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Construcción y evidencia del paso. Después de finalizar una orden, repite PUT y PATCH contra el mismo GUID: deben devolver **409** sin modificar los datos. Las comprobaciones de estado existen en los handlers escritos anteriormente.

Por qué funciona así. La transición Finalizada cambia el estado de negocio. PUT, PATCH y DELETE deben respetarlo y producir 409 si se intenta una operación incompatible.

Verificación concreta. Ninguna modificación se guarda sobre una orden finalizada.

Si algo falla. Si una orden finalizada cambia de estado o desaparece, comprueba la condición Estado antes de escribir.

PASO 2.2.20 — Bloquear DELETE y devolver conflicto 409

Resultado de aprendizaje. Bloquear DELETE y devolver conflicto 409.

Acción. En DELETE bloquea también la eliminación de una orden finalizada devolviendo 409.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Construcción y evidencia del paso. Aplica la misma prueba a DELETE: un recurso `Finalizada` no debe eliminarse; devuelve 409 y un GET posterior debe encontrarlo todavía.

Por qué funciona así. La transición Finalizada cambia el estado de negocio. PUT, PATCH y DELETE deben respetarlo y producir 409 si se intenta una operación incompatible.

Verificación concreta. Un recurso finalizado permanece en la base tras DELETE fallido.

Si algo falla. Si una orden finalizada cambia de estado o desaparece, comprueba la condición Estado antes de escribir.

PASO 2.2.21 — Escribir pruebas HTTP POST y Location

Resultado de aprendizaje. Escribir pruebas HTTP POST y Location.

Acción. Añade EscrituraTests con POST, verifica 201, Location y GET posterior del recurso creado.

Archivo de trabajo: **`tests/AceriaApp.Api.Tests/EscrituraTests.cs`**.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Modelo;
using Xunit;

namespace AceriaApp.Api.Tests;

public class EscrituraTests
{
    // PASO 2.2.21: primera prueba de escritura.
    [Fact]
    public async Task CrearYConsultar_Es201Y200()
    {
        using var f = new ApiFactory();
        using var client = f.CreateClient();
        var created = await client.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX", "CHAPA", 10));
        Assert.Equal(HttpStatusCode.Created, created.StatusCode);
        Assert.NotNull(created.Headers.Location);
        var get = await client.GetAsync(created.Headers.Location);
        Assert.Equal(HttpStatusCode.OK, get.StatusCode);
    }
}
```

Construcción y evidencia del paso. Crea `EscrituraTests.cs` y escribe primero el test `CrearYConsultar_Es201Y200`. El archivo de referencia contiene también tests que construirás en los pasos 2.2.22 y 2.2.23; no pegues otra clase con el mismo nombre.

Por qué funciona así. Los tests deben confirmar los efectos de POST, PUT, PATCH, DELETE, la capacidad del taller y el conflicto de estado a través de peticiones HTTP reales.

Verificación concreta. El test confirma creación y posterior consulta por HTTP.

Si algo falla. Si una prueba de escritura pasa sin volver a consultar la orden, no ha demostrado persistencia; añade el GET posterior.

PASO 2.2.22 — Escribir pruebas HTTP PUT, PATCH y DELETE

Resultado de aprendizaje. Escribir pruebas HTTP PUT, PATCH y DELETE.

Acción. En el test del ciclo completo ejecuta PUT, PATCH y DELETE, leyendo el DTO después de cada cambio.

Archivo de trabajo: **`tests/AceriaApp.Api.Tests/EscrituraTests.cs`**.

```
// Añade dentro de EscrituraTests, sin crear otra clase:
[Fact]
public async Task EditarYEliminar_CambiaPersistencia()
{
    using var f = new ApiFactory();
    using var c = f.CreateClient();
    var made = await c.PostAsJsonAsync("/api/v1/ordenes",
        new CrearOrdenRequest("ACX", "CHAPA", 10));
    var dto = await made.Content.ReadFromJsonAsync<OrdenDto>();
```

```
    Assert.NotNull(dto);
    var put = await c.PutAsJsonAsync("/api/v1/ordenes/" + dto.Id,
        new CrearOrdenRequest("ACX2", "CHAPA", 11));
    Assert.Equal(HttpStatusCode.NoContent, put.StatusCode);
    var patch = await c.PatchAsJsonAsync(
        "/api/v1/ordenes/" + dto.Id + "/cantidad",
        new ActualizarCantidadRequest(12));
    Assert.Equal(HttpStatusCode.OK, patch.StatusCode);
    var changed = await patch.Content.ReadFromJsonAsync<OrdenDto>();
    Assert.Equal(12, changed?.Cantidad);
    var del = await c.DeleteAsync("/api/v1/ordenes/" + dto.Id);
    Assert.Equal(HttpStatusCode.NoContent, del.StatusCode);
    var gone = await c.GetAsync("/api/v1/ordenes/" + dto.Id);
    Assert.Equal(HttpStatusCode.NotFound, gone.StatusCode);
}
```

Construcción y evidencia del paso. Añade en la **misma clase EscrituraTests** el caso `EditarYEliminar_CambiaPersistencia`. Verifica secuencialmente PUT 204, PATCH 200, DELETE 204 y GET 404 con la orden creada al principio.

Por qué funciona así. Los tests deben confirmar los efectos de POST, PUT, PATCH, DELETE, la capacidad del taller y el conflicto de estado a través de peticiones HTTP reales.

Verificación concreta. Se comprueban 204, 200, 204 y después 404.

Si algo falla. Si una prueba de escritura pasa sin volver a consultar la orden, no ha demostrado persistencia; añade el GET posterior.

PASO 2.2.23 — Probar automáticamente 422 y 409

Resultado de aprendizaje. Probar automáticamente 422 y 409.

Acción. Añade pruebas de capacidad 501 y conflicto tras finalizar; ejecuta todo el proyecto xUnit.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Completa `EscrituraTests` con `ReglaCapacidad_Es422` y `OrdenFinalizada_NoSeEdita409`. Ejecuta `dotnet test tests/AceriaApp.Api.Tests` y espera 7 tests acumulados en la etapa 2.2.

Añade estas dos pruebas dentro de `EscrituraTests` (sin duplicar la clase):

```
[Fact]
public async Task ReglaCapacidad_Es422()
{
    using var f = new ApiFactory();
    var r = await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
        new CrearOrdenRequest("ACX", "CHAPA", 501));
    Assert.Equal(HttpStatusCode.UnprocessableEntity, r.StatusCode);
}

[Fact]
public async Task OrdenFinalizada_NoSeEdita409()
{
    using var f = new ApiFactory();
    using var c = f.CreateClient();
    var made = await c.PostAsJsonAsync("/api/v1/ordenes",
        new CrearOrdenRequest("ACX", "CHAPA", 2));
    var dto = await made.Content.ReadFromJsonAsync<OrdenDto>();
    Assert.NotNull(dto);
    var done = await c.PostAsync(
        "/api/v1/ordenes/" + dto.Id + "/finalizar", null);
    Assert.Equal(HttpStatusCode.OK, done.StatusCode);
    var put = await c.PutAsJsonAsync("/api/v1/ordenes/" + dto.Id,
        new CrearOrdenRequest("ACX", "CHAPA", 3));
```

```
    Assert.Equal(HttpStatusCode.Conflict, put.StatusCode);
}
```

Por qué funciona así. Los tests deben confirmar los efectos de POST, PUT, PATCH, DELETE, la capacidad del taller y el conflicto de estado a través de peticiones HTTP reales.

Verificación concreta. La batería comprueba las respuestas 422 y 409.

Si algo falla. Si una prueba de escritura pasa sin volver a consultar la orden, no ha demostrado persistencia; añade el GET posterior.

PASO 2.2.24 — Reto: comprobar idempotencia y efecto persistente

Resultado de aprendizaje. Reto: comprobar idempotencia y efecto persistente.

Acción. Como reto envía dos PUT idénticos y dos DELETE sobre el mismo Guid. Distingue efecto final de código HTTP.

Referencia para comprobar el resultado: los archivos del proyecto `PR0YECT0/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. El reto de idempotencia es opcional y debe probar **efectos persistentes**: dos PUT idénticos no multiplican órdenes, y repetir DELETE no devuelve la entidad eliminada. Conserva evidencia de ambas respuestas y de los GET posteriores.

Por qué funciona así. La idempotencia habla del efecto final: repetir PUT no multiplica los recursos, y repetir DELETE no restaura lo eliminado. Las respuestas HTTP pueden diferir.

Verificación concreta. El estado del recurso no se duplica y el segundo DELETE no lo restaura.

Si algo falla. Si se crean órdenes duplicadas al repetir PUT, se está usando Add en lugar de actualizar la entidad encontrada.

Caso práctico completo — PATCH, eliminación e idempotencia: observar el estado final

Conserva la orden creada en el caso anterior. Ejecuta PATCH sobre `/api/v1/ordenes/{id}/cantidad` con el cuerpo:

```
{"cantidad":15}
```

Comprueba que el código es 200 y que la representación devuelta contiene Cantidad=15, Cliente=ACX-002 y Material=CHAPA-5MM. Esto demuestra que PATCH no ha reconstruido el objeto perdiendo otras propiedades. En el método C# correspondiente solo se reasigna `orden.Cantidad`; no se realiza un `new Orden` ni un `db.Ordenes.Add`.

A continuación ejecuta DELETE con el mismo Guid. El servidor localiza la orden, ejecuta `Remove`, confirma `SaveChangesAsync` y devuelve 204. Envía después GET del mismo identificador: debe producir 404. El recurso ya no existe, aunque la ruta como plantilla siga registrada en la API. Si vuelves a ejecutar DELETE, puede responder 404 porque la orden ya no se encuentra. Esto no contradice la idempotencia de DELETE: el estado final sigue siendo ausencia del recurso.

Realiza una segunda secuencia para reproducir conflicto de estado. Crea otra orden, ejecuta POST sobre `/{id}/finalizar` y después intenta actualizarla con PUT o PATCH. La operación debe devolver 409 sin modificar los datos. La política del taller comprueba Estado=Finalizada antes de escribir; el problema no es que el JSON sea inválido, sino que esa transición ya no está permitida. En otra petición distinta envía Cantidad=501: aunque el rango de valores del contrato admite 501, la capacidad del taller está limitada a 500 y devuelve 422.

Reto final: repite exactamente el mismo PUT sobre una orden pendiente dos veces. Comprueba que el cliente, material y cantidad quedan iguales. La idempotencia se entiende observando el efecto final, no contando solamente códigos 204.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`POST /api/v1/ordenes`	201 + Location
`PUT /{id}`	204
`PATCH /{id}/cantidad`	200
`DELETE /{id}`	204 / 404
`POST /{id}/finalizar`	200 / 409
`Cantidad 501`	422

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.3 — Validación efectiva de peticiones con FluentValidation

Proyecto comprobado: `PROYECTO/2.3-validacion-peticiones`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.3.1 — Comprobar endpoints anteriores y localizar sus contratos

Resultado de aprendizaje. Comprobar endpoints anteriores y localizar sus contratos.

Acción. Abre el proyecto del punto 2.2 y comprueba GET, POST, PUT, PATCH y DELETE antes de modificarlo.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Parte del punto 2.2 funcionando: crea una orden, actualízala y consulta el estado final. Los nuevos validadores no deben cambiar el contrato de los GET ni la capacidad del negocio.

Por qué funciona así. La deserialización de un cuerpo JSON y la validación semántica no son lo mismo. Primero el framework interpreta el mensaje y después se comprueban las reglas de datos.

Verificación concreta. Los tests de escritura siguen pasando.

Si algo falla. Si JSON roto produce un error inesperado, revisa Content-Type y la estructura del cuerpo antes del validador.

PASO 2.3.2 — Distinguir JSON incorrecto, semántica y negocio

Resultado de aprendizaje. Distinguir JSON incorrecto, semántica y negocio.

Acción. Envía tres cuerpos distintos: JSON roto, cantidad 0 y cantidad 501; anota si debe actuar el framework, el validador o el negocio.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Distingue tres fronteras: JSON mal formado (ASP.NET Core), cantidad fuera de rango semántico (FluentValidation) y cantidad que supera la capacidad admitida (regla del manejador). Anota el código y la causa antes de cambiar el código.

Por qué funciona así. La deserialización de un cuerpo JSON y la validación semántica no son lo mismo. Primero el framework interpreta el mensaje y después se comprueban las reglas de datos.

Verificación concreta. Distingues sintaxis, semántica y regla de capacidad.

Si algo falla. Si JSON roto produce un error inesperado, revisa Content-Type y la estructura del cuerpo antes del validador.

PASO 2.3.3 — Instalar FluentValidation.DependencyInjectionExtensions

Resultado de aprendizaje. Instalar FluentValidation.DependencyInjectionExtensions.

Acción. Instala FluentValidation.DependencyInjectionExtensions compatible con .NET 8 en el proyecto de API.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Instala `FluentValidation.DependencyInjectionExtensions` en la API y comprueba las referencias de NuGet. El paquete por sí solo no valida peticiones; hacen falta tipos validadores, DI y filtros conectados.

Por qué funciona así. La deserialización de un cuerpo JSON y la validación semántica no son lo mismo. Primero el framework interpreta el mensaje y después se comprueban las reglas de datos.

Verificación concreta. AbstractValidator y AddValidatorsFromAssemblyContaining se resuelven.

Si algo falla. Si JSON roto produce un error inesperado, revisa Content-Type y la estructura del cuerpo antes del validador.

PASO 2.3.4 — Crear CrearOrdenValidator y reglas de Cliente

Resultado de aprendizaje. Crear CrearOrdenValidator y reglas de Cliente.

Acción. Crea Validadores.cs con CrearOrdenValidator y reglas RuleFor de Cliente con NotEmpty y MaximumLength(100).

Archivo de trabajo: **`src/AceriaApp.Api/Validacion/Validadores.cs`**.

```
using AceriaApp.Api.Contratos;
using FluentValidation;

namespace AceriaApp.Api.Validacion;

public sealed class CrearOrdenValidator : AbstractValidator<CrearOrdenRequest>
{
    public CrearOrdenValidator()
    {
        RuleFor(x => x.Cliente).NotEmpty().MaximumLength(100);
    }
}
```

Secuencia de construcción. Crea `src/AceriaApp.Api/Validacion/Validadores.cs` con el validador inicial de Cliente. En 2.3.5 ampliarás su constructor para Material y Cantidad. No copies aún el validador de PATCH.

Por qué funciona así. FluentValidation expresa reglas sobre los contratos de entrada y devuelve errores por campo; no debe confundirse un record con el validador que lo inspecciona.

Verificación concreta. Un cliente vacío infringe una regla semántica.

Si algo falla. Si RuleFor no compila, revisa FluentValidation y el using. Comprueba la diferencia entre NotEmpty y límites numéricos.

PASO 2.3.5 — Definir Material y rango válido de Cantidad

Resultado de aprendizaje. Definir Material y rango válido de Cantidad.

Acción. En ese validador incorpora Material obligatorio y Cantidad InclusiveBetween(1,10000).

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Código de este paso. Añade las reglas dentro del mismo constructor, después de la de Cliente. El rango semántico `1..10000` es diferente del límite de capacidad de negocio `500` evaluado en el endpoint.

```
// Añade estas dos líneas al constructor de CrearOrdenValidator:
RuleFor(x => x.Material).NotEmpty().MaximumLength(100);
RuleFor(x => x.Cantidad).InclusiveBetween(1, 10000);
```

Por qué funciona así. FluentValidation expresa reglas sobre los contratos de entrada y devuelve errores por campo; no debe confundirse un record con el validador que lo inspecciona.

Verificación concreta. La cantidad 0 o 10001 es inválida; 501 cumple el rango.

Si algo falla. Si RuleFor no compila, revisa FluentValidation y el using. Comprueba la diferencia entre NotEmpty y límites numéricos.

PASO 2.3.6 — Crear ActualizarCantidadValidator para PATCH

Resultado de aprendizaje. Crear ActualizarCantidadValidator para PATCH.

Acción. Añade ActualizarCantidadValidator para el request de PATCH y su regla de cantidad.

Archivo de trabajo: **`src/AceriaApp.Api/Validacion/Validadores.cs`**.

```
// Añade al final de Validadores.cs, fuera de CrearOrdenValidator:
public sealed class ActualizarCantidadValidator
    : AbstractValidator<ActualizarCantidadRequest>
{
    public ActualizarCantidadValidator()
    {
        RuleFor(x => x.Cantidad).InclusiveBetween(1, 10000);
    }
}
```

Secuencia de construcción. No sustituye a `CrearOrdenValidator`: es una **segunda clase** del mismo archivo `Validadores.cs`, destinada a PATCH. Ambas clases deben compilar juntas.

Por qué funciona así. FluentValidation expresa reglas sobre los contratos de entrada y devuelve errores por campo; no debe confundirse un record con el validador que lo inspecciona.

Verificación concreta. PATCH comparte el criterio semántico del campo Cantidad.

Si algo falla. Si RuleFor no compila, revisa FluentValidation y el using. Comprueba la diferencia entre NotEmpty y límites numéricos.

PASO 2.3.7 — Ejecutar validadores y estudiar ValidationResult

Resultado de aprendizaje. Ejecutar validadores y estudiar ValidationResult.

Acción. Instancia los validadores en un test sencillo y observa IsValid y Errors con peticiones válidas e inválidas.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Crea una pequeña prueba de exploración con `CrearOrdenValidator`: compara una instancia válida y otra con `Cantidad=0`. Observa `IsValid` y `Errors`; estas comprobaciones aisladas no sustituyen la prueba HTTP del filtro conectado.

Por qué funciona así. FluentValidation expresa reglas sobre los contratos de entrada y devuelve errores por campo; no debe confundirse un record con el validador que lo inspecciona.

Verificación concreta. Comprendes ValidationResult antes de conectar HTTP.

Si algo falla. Si RuleFor no compila, revisa FluentValidation y el using. Comprueba la diferencia entre NotEmpty y límites numéricos.

PASO 2.3.8 — Crear ValidationFilter genérico de Minimal API

Resultado de aprendizaje. Crear ValidationFilter genérico de Minimal API.

Acción. Crea ValidationFilter.cs e implementa IEndpointFilter y su InvokeAsync.

Archivo de trabajo: **`src/AceriaApp.Api/Validacion/ValidationFilter.cs`**.

```
using FluentValidation;

namespace AceriaApp.Api.Validacion;

public sealed class ValidationFilter<T>(IValidator<T> validator)
    : IEndpointFilter where T : class
{
    public async ValueTask<object?> InvokeAsync(
        EndpointFilterInvocationContext context, EndpointFilterDelegate next)
    {
        var request = context.Arguments.OfType<T>().FirstOrDefault();
        if (request is null)
            return Results.ValidationProblem(
                new Dictionary<string,string[]> {
                    ["body"] = new[] { "Falta el cuerpo." }
                });
        var result = await validator.ValidateAsync(
            request, context.HttpContext.RequestAborted);
        if (!result.IsValid)
        {
            var errors = result.Errors.GroupBy(x => x.PropertyName)
                .ToDictionary(x => x.Key,
                    x => x.Select(e => e.ErrorMessage).ToArray());
            return Results.ValidationProblem(errors);
        }
        return await next(context);
    }
}
```

Secuencia de construcción. En `Validacion/ValidationFilter.cs` implementa el filtro completo para que compile; los pasos 2.3.9–2.3.13 descompondrán sus decisiones. No repitas el código en esas actividades; depúralo y observa dónde sale el flujo.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. El filtro recibe EndpointFilterInvocationContext y EndpointFilterDelegate.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.9 — Obtener la petición de EndpointFilterInvocationContext

Resultado de aprendizaje. Obtener la petición de EndpointFilterInvocationContext.

Acción. En InvokeAsync usa context.Arguments.OfType().FirstOrDefault para localizar el contrato de la petición.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Construcción y evidencia del paso. En `ValidationFilter<T>.InvokeAsync`, coloca un punto de interrupción en `context.Arguments.OfType<T>()`. Identifica por qué el request se busca por tipo y qué respuesta debe emitirse si no se encuentra.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. Un request ausente no provoca NullReferenceException.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.10 — Ejecutar ValidateAsync y respetar RequestAborted

Resultado de aprendizaje. Ejecutar ValidateAsync y respetar RequestAborted.

Acción. Invoca ValidateAsync pasando RequestAborted desde HttpContext para respetar cancelaciones.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Construcción y evidencia del paso. Depura la invocación `ValidateAsync(request, context.HttpContext.RequestAborted)`. Comprueba que usa el request real y que la cancelación proviene del contexto HTTP; no crees un token global.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. No continúa una operación que el cliente ha cancelado.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.11 — Agrupar errores por campo con LINQ

Resultado de aprendizaje. Agrupar errores por campo con LINQ.

Acción. Agrupa result.Errors por PropertyName y construye un diccionario string[] de mensajes.

Archivo de trabajo: **`src/AceriaApp.Api/Validacion/ValidationFilter.cs`**.

```
// Dentro de InvokeAsync, en la rama !result.IsValid:
var errors = result.Errors.GroupBy(x => x.PropertyName)
```

```
    .ToDictionary(x => x.Key,
        x => x.Select(e => e.ErrorMessage).ToArray());
return Results.ValidationProblem(errors);
```

Secuencia de construcción. Este fragmento **ya forma parte** de `ValidationFilter.cs` creado en 2.3.8. Utilízalo para analizar la transformación de `ValidationFailure` en el diccionario de errores; no pegues otra clase.

Construcción y evidencia del paso. Separa la agrupación por `PropertyName` de la conversión a `string[]`. Envía dos datos inválidos a la vez y comprueba que `errors` tiene una clave por propiedad y un array de mensajes por clave.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. Cada propiedad tiene su colección de errores.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.12 — Responder mediante Results.ValidationProblem

Resultado de aprendizaje. Responder mediante Results.ValidationProblem.

Acción. Ante fallo devuelve Results.ValidationProblem(errors) sin invocar el manejador siguiente.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Construcción y evidencia del paso. Cuando `IsValid=false`, devuelve `Results.ValidationProblem(errors)` y comprueba 400 con JSON de errores. Verifica con un GET posterior que un POST inválido no ha insertado la orden.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. La respuesta incluye status 400 y el campo errors.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

Caso práctico completo — Comprobar que el filtro realmente impide entrar al manejador

Arranca el proyecto 2.3 y abre `ValidationFilter.cs` y `Program.cs` al mismo tiempo. Coloca un breakpoint dentro de `InvokeAsync` y otro en la primera instrucción del manejador POST que crea la orden. Prepara una petición con Cliente vacío y los demás campos válidos. El deserializador de ASP.NET Core construirá CrearOrdenRequest porque el JSON tiene una estructura compatible; a continuación se ejecutará el filtro al que se asoció `AddEndpointFilter`.

Inspecciona el valor `request` en el depurador y observa cómo `ValidateAsync` devuelve `IsValid=false`. El filtro agrupa los `ValidationFailure` por nombre de propiedad y pasa el diccionario a `Results.ValidationProblem`. El manejador POST no debe ejecutarse, ni tampoco debe llamarse `SaveChangesAsync`. La respuesta HTTP contiene un 400 y un objeto JSON con la propiedad `errors`.

Repite con un cuerpo válido. El filtro volverá a ejecutarse, pero esta vez `IsValid` será verdadero y se llamará a `await next(context)`. El breakpoint del manejador POST debe alcanzarse. La orden se persiste y el servidor devuelve 201. Esta comparación entre las dos trazas es la demostración más importante del punto: crear una clase validadora y registrarla en DI no basta; hay que conectarla al endpoint que recibe la petición.

Comprueba después PUT y PATCH. El primero utiliza el contrato CrearOrdenRequest y el segundo ActualizarCantidadRequest. Cada uno debe utilizar el tipo genérico correcto al añadir el filtro. Un error común consiste en filtrar solamente POST y creer que todos los verbos quedan automáticamente protegidos. En ese caso PUT o PATCH aceptarían datos que POST rechaza.

Verificación: enviar Cantidad=0 debe devolver 400 con una entrada en errors; Cantidad=501 debe producir 422 porque es una regla del negocio.

PASO 2.3.13 — Permitir next(context) solo cuando IsValid

Resultado de aprendizaje. Permitir next(context) solo cuando IsValid.

Acción. Si IsValid es verdadero retorna await next(context); razona qué sucede al omitirlo.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Construcción y evidencia del paso. En la rama válida debe ejecutarse `next(context)`; si falta, el filtro intercepta todas las peticiones sin permitir el POST real. Contrasta con una orden válida y su `Location`.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. El endpoint se ejecuta solo cuando la validación pasa.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.14 — Registrar validadores en el contenedor DI

Resultado de aprendizaje. Registrar validadores en el contenedor DI.

Acción. Antes de builder.Build registra los validadores mediante AddValidatorsFromAssemblyContaining().

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
using FluentValidation;
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>();
```

Construcción y evidencia del paso. Registra `AddValidatorsFromAssemblyContaining<CrearOrdenValidator>()` antes de `builder.Build()`. Verifica que DI resuelve `IValidator<CrearOrdenRequest>` y `IValidator<ActualizarCantidadRequest>`.

Por qué funciona así. El registro de validadores en DI no ejecuta la validación. AddEndpointFilter debe asociarse a cada POST, PUT o PATCH que acepte ese contrato.

Verificación concreta. El filtro puede resolver IValidator.

Si algo falla. Si se acepta una petición inválida, puede faltar AddEndpointFilter aunque los validadores estén registrados en DI.

PASO 2.3.15 — Conectar filtro de CrearOrdenRequest al POST

Resultado de aprendizaje. Conectar filtro de CrearOrdenRequest al POST.

Acción. Añade AddEndpointFilter<ValidationFilter>() al MapPost real.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
.WithName("CrearOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()

;
```

Construcción y evidencia del paso. En el **POST existente**, añade `AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()`. Ejecuta un caso válido (201) y otro con Cliente vacío (400). No crees otra ruta POST.

Por qué funciona así. El registro de validadores en DI no ejecuta la validación. AddEndpointFilter debe asociarse a cada POST, PUT o PATCH que acepte ese contrato.

Verificación concreta. POST con Cliente vacío devuelve 400 en vez de guardar una fila.

Si algo falla. Si se acepta una petición inválida, puede faltar AddEndpointFilter aunque los validadores estén registrados en DI.

PASO 2.3.16 — Conectar filtro de CrearOrdenRequest al PUT

Resultado de aprendizaje. Conectar filtro de CrearOrdenRequest al PUT.

Acción. Añade el mismo filtro de CrearOrdenRequest al MapPut.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Construcción y evidencia del paso. Aplica el mismo filtro al PUT existente y utiliza una orden `Pendiente` real. Envía un request inválido y consulta por GET: debe devolver 400 sin modificar la orden.

Código que se incorpora en este paso:

```
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>();
```

Por qué funciona así. El registro de validadores en DI no ejecuta la validación. AddEndpointFilter debe asociarse a cada POST, PUT o PATCH que acepte ese contrato.

Verificación concreta. PUT inválido no modifica la orden almacenada.

Si algo falla. Si se acepta una petición inválida, puede faltar AddEndpointFilter aunque los validadores estén registrados en DI.

PASO 2.3.17 — Conectar filtro de ActualizarCantidadRequest al PATCH

Resultado de aprendizaje. Conectar filtro de ActualizarCantidadRequest al PATCH.

Acción. Añade AddEndpointFilter<ValidationFilter>() al MapPatch.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
.WithName("CambiarCantidad").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()

;
```

Construcción y evidencia del paso. Asocia `ValidationFilter<ActualizarCantidadRequest>` al PATCH de `/cantidad`; comprueba el caso inválido y el válido. Los tres filtros deben convivir sin alterar los GET.

Por qué funciona así. El registro de validadores en DI no ejecuta la validación. AddEndpointFilter debe asociarse a cada POST, PUT o PATCH que acepte ese contrato.

Verificación concreta. PATCH con Cantidad=0 devuelve 400.

Si algo falla. Si se acepta una petición inválida, puede faltar AddEndpointFilter aunque los validadores estén registrados en DI.

PASO 2.3.18 — Enviar POST incorrecto y comprobar status 400

Resultado de aprendizaje. Enviar POST incorrecto y comprobar status 400.

Acción. Envía un POST con Material vacío y comprueba código, Content-Type y ausencia de inserción.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Construcción y evidencia del paso. Desde el archivo `.http` envía `Material=""`. Espera HTTP 400 y un cuerpo `ValidationProblem`; comprueba que no se ha insertado nada en la base persistente.

Por qué funciona así. Un 400 por datos fuera del rango, un 422 por la capacidad y un 409 por el estado de una orden pertenecen a causas diferentes que el cliente debe distinguir.

Verificación concreta. El código es 400 y la base permanece sin cambios.

Si algo falla. Si aparece 500 en vez de 400/422/409, revisa qué capa lanza la excepción y transforma el error de forma adecuada.

PASO 2.3.19 — Inspeccionar la propiedad errors del Problem Details

Resultado de aprendizaje. Inspeccionar la propiedad errors del Problem Details.

Acción. Examina el JSON Problem Details y localiza en errors el nombre de la propiedad rechazada.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Construcción y evidencia del paso. Revisa el `Content-Type` y la propiedad `errors`. Identifica el nombre de campo que devuelve FluentValidation y no confundas el detalle de errores de contrato con `Results.Problem` del negocio.

Por qué funciona así. Un 400 por datos fuera del rango, un 422 por la capacidad y un 409 por el estado de una orden pertenecen a causas diferentes que el cliente debe distinguir.

Verificación concreta. El cliente puede mostrar el mensaje bajo el campo correcto.

Si algo falla. Si aparece 500 en vez de 400/422/409, revisa qué capa lanza la excepción y transforma el error de forma adecuada.

PASO 2.3.20 — Probar límite 10001 frente a la capacidad 501

Resultado de aprendizaje. Probar límite 10001 frente a la capacidad 501.

Acción. Compara un POST de cantidad 10001 con otro de 501 y anota el origen del error.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Construcción y evidencia del paso. Compara dos peticiones que difieren solo en cantidad: `10001` debe fallar en el validador con 400; `501` supera la regla de negocio pero está dentro del rango semántico, por eso devuelve 422.

Por qué funciona así. Un 400 por datos fuera del rango, un 422 por la capacidad y un 409 por el estado de una orden pertenecen a causas diferentes que el cliente debe distinguir.

Verificación concreta. El primero devuelve 400, el segundo 422.

Si algo falla. Si aparece 500 en vez de 400/422/409, revisa qué capa lanza la excepción y transforma el error de forma adecuada.

PASO 2.3.21 — Reproducir 409 con orden ya finalizada

Resultado de aprendizaje. Reproducir 409 con orden ya finalizada.

Acción. Crea, finaliza e intenta editar una orden: no es un fallo semántico, sino un conflicto de estado.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Crea, finaliza y vuelve a editar una orden: debe resultar 409, no 400. El estado persistente condiciona la operación aunque el JSON enviado sea correcto.

Por qué funciona así. Un 400 por datos fuera del rango, un 422 por la capacidad y un 409 por el estado de una orden pertenecen a causas diferentes que el cliente debe distinguir.

Verificación concreta. Se recibe 409 y los datos no cambian.

Si algo falla. Si aparece 500 en vez de 400/422/409, revisa qué capa lanza la excepción y transforma el error de forma adecuada.

PASO 2.3.22 — Crear pruebas de validación mediante HTTP

Resultado de aprendizaje. Crear pruebas de validación mediante HTTP.

Acción. Crea ValidacionHttpTests con peticiones reales para Cliente, Material, Cantidad y política de negocio.

Archivo de trabajo: **`tests/AceriaApp.Api.Tests/ValidacionHttpTests.cs`**.

```
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;
using AceriaApp.Api.Contratos;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.3.22: se verifica el filtro ACTIVO, no su mera existencia.
public class ValidacionHttpTests
{
    [Theory]
    [InlineData("","CHAPA",10)]
    [InlineData("ACX","",10)]
    [InlineData("ACX","CHAPA",0)]
    public async Task DatosInvalidos_Es400ConErrors(string cliente,string material,int cantidad)
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest(cliente,material,cantidad));
        Assert.Equal(HttpStatusCode.BadRequest,r.StatusCode);
        using var json=JsonDocument.Parse(await r.Content.ReadAsStringAsync());
        Assert.True(json.RootElement.TryGetProperty("errors",out _));
    }
    [Fact] public async Task CantidadMasDe10000_Es400()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10001));
        Assert.Equal(HttpStatusCode.BadRequest,r.StatusCode);
    }
    [Fact] public async Task ReglaDeNegocio_501_Es422()
    {
        using var f=new ApiFactory();
```

```
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",501));
        Assert.Equal(HttpStatusCode.UnprocessableEntity,r.StatusCode);
    }
}
```

Secuencia de construcción. Escribe el archivo completo mostrado, con los cinco casos (tres de la teoría parametrizada y dos hechos independientes). Ejecuta las pruebas después de conectar los filtros de POST, PUT y PATCH.

Por qué funciona así. Las pruebas HTTP recorren el pipeline completo y comprueban que los fallos se comunican como Problem Details sin alterar los datos.

Verificación concreta. Las respuestas se prueban a través del pipeline completo.

Si algo falla. Si el test solo instancia un record, no prueba HTTP; usa WebApplicationFactory y examina errors del JSON.

PASO 2.3.23 — Verificar que los GET siguen sin regresiones

Resultado de aprendizaje. Verificar que los GET siguen sin regresiones.

Acción. Ejecuta los tests GET y de escritura anteriores, además de las nuevas pruebas de validación.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Ejecuta `dotnet test tests/AceriaApp.Api.Tests`: deben pasar 12 tests acumulados. Comprueba que continúan los casos GET, escritura y validación; un test nuevo correcto no compensa una regresión.

Por qué funciona así. Las pruebas HTTP recorren el pipeline completo y comprueban que los fallos se comunican como Problem Details sin alterar los datos.

Verificación concreta. El filtrado no rompe las rutas que ya funcionaban.

Si algo falla. Si el test solo instancia un record, no prueba HTTP; usa WebApplicationFactory y examina errors del JSON.

PASO 2.3.24 — Reto: personalizar mensajes y comprobar errores

Resultado de aprendizaje. Reto: personalizar mensajes y comprobar errores.

Acción. Reto: incorpora WithMessage a una regla y comprueba desde un test HTTP el texto dentro de errors.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. El reto de `WithMessage` es opcional. Si lo realizas, cambia una regla concreta, ejecuta una petición inválida y verifica el mensaje dentro de la clave adecuada de `errors`, sin modificar los códigos de respuesta.

Por qué funciona así. Las pruebas HTTP recorren el pipeline completo y comprueban que los fallos se comunican como Problem Details sin alterar los datos.

Verificación concreta. El mensaje devuelto es específico y mantiene status 400.

Si algo falla. Si el test solo instancia un record, no prueba HTTP; usa WebApplicationFactory y examina errors del JSON.

Caso práctico completo — Del error 400 al 409: tres pruebas con causas diferentes

Para comprobar la separación de responsabilidades, prepara primero un POST de cantidad 10.001. Es un número entero válido desde el punto de vista sintáctico, pero infringe la regla `InclusiveBetween(1,10000)`. Por tanto, FluentValidation genera un error semántico y el filtro devuelve 400. Inspecciona el campo `errors`: debe identificar Cantidad. No necesitas consultar SQL Server para determinar que un valor está fuera de ese intervalo.

Envía después exactamente el mismo cuerpo, cambiando Cantidad a 501. El validador semántico deja pasar el valor, porque 501 pertenece al rango de entrada. El manejador POST comprueba entonces la capacidad del taller y decide responder 422. La decisión podría depender en el futuro de inventario, estado de las máquinas o política comercial; por esa razón no es idéntica a una comprobación de formato. Este es un contrato elegido para el ejercicio, no una regla universal que obligue a todas las APIs a usar 422.

La tercera prueba necesita un recurso persistido. Crea una orden con cantidad 10, recupera su Guid, ejecútale `/finalizar` y después envía un PUT de modificación. El servidor devuelve 409 porque el recurso está en un estado incompatible con la modificación. Un 409 no significa que el tipo `int` de Cantidad sea inválido: significa que el recurso existente no admite la operación solicitada.

Escribe pruebas en `ValidacionHttpTests.cs` que cubran las tres situaciones. Usa `WebApplicationFactory` para hacer peticiones HTTP reales y comprobar el código que recibe un consumidor. La prueba de 400 debe revisar `errors`; las de 422 y 409 comprueban la distinción de política y estado.

Control de regresión: después de añadir filtros, vuelve a ejecutar GET de lista y detalle. Ninguna regla de validación debería convertir una lectura válida en un fallo de contrato.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`Cliente vacío`	400 + errors
`Cantidad 10001`	400 + errors
`Cantidad 501`	422
`Orden finalizada`	409

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.4 — Documento OpenAPI generado y comprobado con Swagger

Proyecto comprobado: `PROYECTO/2.4-documentacion-openapi`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.4.1 — Comprobar el contrato REST que se va a documentar

Resultado de aprendizaje. Comprobar el contrato REST que se va a documentar.

Acción. Abre la etapa 2.3 y anota sus rutas, DTOs, estados 200/201/204/400/409/422 antes de documentarla.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.4-documentacion-openapi` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Antes de documentar, haz inventario de las siete operaciones de órdenes y de sus respuestas reales. Los metadatos que declares después deben coincidir con el servidor, no con resultados imaginados.

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. La futura documentación reflejará endpoints ya comprobados.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.2 — Instalar Swashbuckle.AspNetCore compatible con .NET 8

Resultado de aprendizaje. Instalar Swashbuckle.AspNetCore compatible con .NET 8.

Acción. Instala Swashbuckle.AspNetCore 6.6.2 para .NET 8 en el proyecto API, no solo en tests.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.4-documentacion-openapi` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Instala `Swashbuckle.AspNetCore` 6.6.2 en la API de esta etapa. Una instalación correcta no publica ningún documento hasta registrar servicios y middleware.

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. AddSwaggerGen y UseSwagger están disponibles.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.3 — Crear carpeta Documentacion y extensiones de configuración

Resultado de aprendizaje. Crear carpeta Documentacion y extensiones de configuración.

Acción. Crea Documentacion/DocumentacionExtensions.cs y una clase estática DocumentacionExtensions.

Archivo de trabajo: **`src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`**.

```
// Crea src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs:
namespace AceriaApp.Api.Documentacion;

public static class DocumentacionExtensions
{
    // En 2.4.4 se añadirá AddDocumentacion.
    // En 2.4.7 se añadirá UseDocumentacion.
}
```

Secuencia de construcción. Crea una clase estática vacía para organizar el registro y la publicación de Swagger. No añadas todavía `EjemploCrearOrden`, que corresponde al paso 2.4.16.

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. La configuración OpenAPI queda separada de Program.cs.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.4 — Registrar AddEndpointsApiExplorer y AddSwaggerGen

Resultado de aprendizaje. Registrar AddEndpointsApiExplorer y AddSwaggerGen.

Acción. En AddDocumentacion registra AddEndpointsApiExplorer y AddSwaggerGen con opciones.

Archivo de trabajo: **`src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`**.

```
// Dentro de DocumentacionExtensions:
public static IServiceCollection AddDocumentacion(
    this IServiceCollection services)
{
    services.AddEndpointsApiExplorer();
    services.AddSwaggerGen();
    return services;
}
```

Secuencia de construcción. Añade un método de extensión que registra el explorador de endpoints y el generador. Se necesitan los paquetes del paso 2.4.2 y los `using` de ASP.NET Core. En 2.4.5 configurarás las opciones de `AddSwaggerGen`.

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. Los servicios descubren los endpoints Minimal API.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.5 — Configurar SwaggerDoc con información y versión v1

Resultado de aprendizaje. Configurar SwaggerDoc con información y versión v1.

Acción. Configura SwaggerDoc v1 con título, versión, descripción y contacto de AceriaApp.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Código de este paso. En el mismo archivo agrega `using Microsoft.OpenApi.Models;`. El documento se llama `v1`: eso no crea automáticamente otra versión de la API.

```
// Sustituye services.AddSwaggerGen() dentro de AddDocumentacion:
services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "AceriaApp — Órdenes de producción",
        Version = "v1",
        Description = "API REST educativa creada con ASP.NET Core 8.",
        Contact = new OpenApiContact
        {
            Name = "Equipo AceriaApp",
            Email = "equipo@acerialocal"
        }
    });
});
```

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. La sección info del JSON muestra esos valores.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.6 — Conectar AddDocumentacion en Program.cs

Resultado de aprendizaje. Conectar AddDocumentacion en Program.cs.

Acción. En Program.cs llama builder.Services.AddDocumentacion() antes de builder.Build().

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
builder.Services.AddDocumentacion();
```

Construcción y evidencia del paso. En `Program.cs` añade `builder.Services.AddDocumentacion()` **antes** de `builder.Build()`, conservando el registro del DbContext y FluentValidation. El método pertenece al archivo que estás construyendo en 2.4.3–2.4.5.

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. Se registra el generador en el contenedor.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.7 — Configurar UseSwagger y Swagger UI en Development

Resultado de aprendizaje. Configurar UseSwagger y Swagger UI en Development.

Acción. En UseDocumentacion coloca UseSwagger y UseSwaggerUI dentro de la condición IsDevelopment.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Código de este paso. Incorpora el método en la misma clase estática. La restricción a `Development` debe ser comprobada por un test posterior.

```
// Añade a la misma clase DocumentacionExtensions:
public static WebApplication UseDocumentacion(this WebApplication app)
{
    if (app.Environment.IsDevelopment())
    {
        app.UseSwagger();
        app.UseSwaggerUI(o =>
            o.SwaggerEndpoint("/swagger/v1/swagger.json", "AceriaApp v1"));
    }
    return app;
}
```

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. Swagger no se publica automáticamente en Production.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.8 — Conectar UseDocumentacion y abrir /swagger

Resultado de aprendizaje. Conectar UseDocumentacion y abrir /swagger.

Acción. Llama app.UseDocumentacion() después de construir la aplicación y abre /swagger desde Visual Studio.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
app.UseDocumentacion(); // PASO 2.4.4
```

Construcción y evidencia del paso. Coloca `app.UseDocumentacion()` antes de `app.Run()`; abre `/swagger` con el servidor en Development. Verifica que Swagger UI muestra las rutas existentes.

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. La interfaz presenta los endpoints del proyecto.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.9 — Descargar el JSON real /swagger/v1/swagger.json

Resultado de aprendizaje. Descargar el JSON real /swagger/v1/swagger.json.

Acción. Descarga /swagger/v1/swagger.json como texto JSON, no una cadena YAML fabricada.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Construcción y evidencia del paso. Solicita `GET /swagger/v1/swagger.json` a la API levantada. Guarda el cuerpo JSON devuelto por el servidor; no sustituyas la prueba por un archivo estático escrito a mano.

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. La petición devuelve 200 y el documento empieza por openapi.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.10 — Identificar info, paths, components y schemas

Resultado de aprendizaje. Identificar info, paths, components y schemas.

Acción. Busca las secciones info, paths y components; identifica las rutas de órdenes y sus verbos.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Construcción y evidencia del paso. Localiza `info.version`, `paths` y `components.schemas` en el documento. Compara las rutas publicadas con las que aparecen en `Program.cs`.

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. GET, POST, PUT, PATCH y DELETE figuran donde corresponde.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.11 — Aplicar WithName y WithTags a los endpoints

Resultado de aprendizaje. Aplicar WithName y WithTags a los endpoints.

Acción. En Program.cs examina WithName y WithTags para agrupar los endpoints de órdenes.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Construcción y evidencia del paso. Revisa los `WithName` y `WithTags` ya usados en los endpoints anteriores. No los dupliques: comprueba su efecto en la agrupación por `Ordenes` de Swagger UI.

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. Los nombres se utilizan también en CreatedAtRoute.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

PASO 2.4.12 — Documentar GET con WithSummary y Produces

Resultado de aprendizaje. Documentar GET con WithSummary y Produces.

Acción. Agrega WithSummary y Produces a los GET y comprueba 200 y 404.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
```

Construcción y evidencia del paso. Añade `WithSummary` y `Produces` a los GET existentes y conserva el cuerpo de cada manejador. Para la colección corresponde 200 con `IEnumerable<OrdenDto>`; para el detalle corresponde 200 con `OrdenDto` y alternativa 404.

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. El documento describe la lectura y la ausencia del recurso.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

Caso práctico completo — Desde MapGet hasta paths: interpretar Swagger JSON

Inicia la API 2.4 en Development y abre la URL `http://localhost:5082/swagger/v1/swagger.json`. Este documento no se guarda a mano en la carpeta docs. Lo produce SwaggerGen cuando la aplicación solicita información sobre los endpoints de ASP.NET Core. Busca la propiedad `info`: debe contener el título de AceriaApp y la versión v1 que registró `SwaggerDoc`.

Continúa con la sección `paths`. Busca `/api/v1/ordenes` y despliega sus operaciones `get` y `post`. Cada operación representa un manejador declarado mediante MapGet o MapPost. La sección muestra los tipos de cuerpo de entrada y las respuestas descritas con `Produces`, `ProducesProblem` y otros metadatos. Una ruta ausente en el documento puede indicar que no está registrada, que la exploración no está configurada o que sus metadatos son incompletos. No lo arregles escribiendo una ruta ficticia en un YAML separado.

Abre a continuación Swagger UI en `/swagger`. La interfaz interpreta el JSON y permite visualizar los endpoints agrupados con `WithTags`. Selecciona GET de colección, ejecuta Try it out y compara el estado 200 con el del archivo HTTP del alumno. Las dos herramientas consumen la misma API; Swagger UI no proporciona un servidor de ejemplo independiente.

En `Program.cs` revisa `WithSummary` y `Produces`. Añade o corrige el resumen del GET y vuelve a descargar el JSON: el metadato cambia porque la fuente de la documentación es el código. Esta relación bidireccional entre implementar y documentar es precisamente el aprendizaje del punto 2.4.

Error habitual: configurar AddSwaggerGen pero olvidar `app.UseDocumentacion()` después de Build. En ese caso los servicios pueden estar registrados y, sin embargo, /swagger devolver 404.

PASO 2.4.13 — Documentar POST con 201 y ValidationProblem 400

Resultado de aprendizaje. Documentar POST con 201 y ValidationProblem 400.

Acción. Documenta POST con Produces(201) y ProducesValidationProblem(400).

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Construcción y evidencia del paso. En el POST añade declaraciones de respuesta 201 y `ValidationProblem(400)` que correspondan al código que ejecuta la etapa 2.3; conserva 422 como regla de capacidad.

Código que se incorpora en este paso:

```
.WithName("CrearOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Crea una orden")
.Produces<OrdenDto>(201)
.ProducesValidationProblem(400)
.ProducesProblem(422);
```

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. Swagger enumera creación y error de contrato.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

PASO 2.4.14 — Documentar PUT y PATCH con códigos alternativos

Resultado de aprendizaje. Documentar PUT y PATCH con códigos alternativos.

Acción. Documenta PUT y PATCH con sus códigos, evitando declarar respuestas inventadas.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Construcción y evidencia del paso. Revisa PUT y PATCH en `Program.cs` y documenta sus respuestas reales: PUT 204, 404, 400, 409 y 422; PATCH 200, 404, 400, 409 y 422. Declara esos metadatos sin cambiar las ramas de ejecución.

Código que se incorpora en este paso:

```
// Al final del MapPut:
.WithSummary("Sustituye campos editables")
.Produces(204).Produces(404)
.ProducesProblem(409)
.ProducesValidationProblem(400)
.ProducesProblem(422);

// Al final del MapPatch:
.WithSummary("Modifica cantidad")
.Produces<OrdenDto>(200).Produces(404)
.ProducesValidationProblem(400)
.ProducesProblem(409)
.ProducesProblem(422);
```

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. La documentación debe coincidir con los Results usados.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

PASO 2.4.15 — Documentar DELETE y el conflicto 409

Resultado de aprendizaje. Documentar DELETE y el conflicto 409.

Acción. Documenta DELETE y el conflicto 409 de órdenes finalizadas.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Construcción y evidencia del paso. Documenta DELETE con 204, 404 y 409, y el endpoint `finalizar` con 200, 404 y 409. Un código anunciado por Swagger debe estar respaldado por una rama de ejecución o un comportamiento del framework.

Código que se incorpora en este paso:

```
// Al final del MapDelete:
.WithSummary("Elimina una orden pendiente")
.Produces(204).Produces(404)
.ProducesProblem(409);

// Al final del MapPost que finaliza una orden:
.WithSummary("Finaliza una orden")
.Produces<OrdenDto>(200).Produces(404)
.ProducesProblem(409);
```

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. El consumidor puede anticipar una eliminación rechazada.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

PASO 2.4.16 — Crear SchemaFilter EjemploCrearOrden

Resultado de aprendizaje. Crear SchemaFilter EjemploCrearOrden.

Acción. Crea EjemploCrearOrden : ISchemaFilter y asigna OpenApiObject para los tres campos del request.

Archivo de trabajo: **`src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`**.

```
// Añade al archivo, fuera de la clase DocumentacionExtensions:
public sealed class EjemploCrearOrden : ISchemaFilter
{
    public void Apply(OpenApiSchema schema, SchemaFilterContext context)
    {
        if (context.Type != typeof(CrearOrdenRequest)) return;
        schema.Example = new OpenApiObject
        {
            ["cliente"] = new OpenApiString("ACX-001"),
            ["material"] = new OpenApiString("CHAPA-3MM"),
            ["cantidad"] = new OpenApiInteger(10)
        };
    }
}
// Dentro de AddSwaggerGen(options => {...}), agrega:
options.SchemaFilter<EjemploCrearOrden>();
```

Secuencia de construcción. Importa `AceriaApp.Api.Contratos`, `Microsoft.OpenApi.Any`, `Microsoft.OpenApi.Models` y `Swashbuckle.AspNetCore.SwaggerGen`. Esta es la última ampliación del archivo que lo deja equivalente a la solución de 2.4.

Por qué funciona así. Un ejemplo JSON ilustra el contrato de entrada. Swagger UI permite usarlo para enviar una petición real, pero no crea datos hasta ejecutar el endpoint.

Verificación concreta. El esquema contiene cliente, material y cantidad ilustrativos.

Si algo falla. Si no aparece example, comprueba SchemaFilter y el tipo CrearOrdenRequest usado en el esquema.

PASO 2.4.17 — Inspeccionar example de CrearOrdenRequest en Swagger

Resultado de aprendizaje. Inspeccionar example de CrearOrdenRequest en Swagger.

Acción. En JSON comprueba components.schemas.CrearOrdenRequest.example y explica que no son datos persistidos.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Construcción y evidencia del paso. Ejecuta la API, solicita el documento JSON real y busca `components.schemas.CrearOrdenRequest.example`. El ejemplo proviene del SchemaFilter, no de una orden almacenada.

Por qué funciona así. Un ejemplo JSON ilustra el contrato de entrada. Swagger UI permite usarlo para enviar una petición real, pero no crea datos hasta ejecutar el endpoint.

Verificación concreta. El ejemplo aparece en la sección reusable de esquemas.

Si algo falla. Si no aparece example, comprueba SchemaFilter y el tipo CrearOrdenRequest usado en el esquema.

PASO 2.4.18 — Ejecutar un POST desde Swagger UI

Resultado de aprendizaje. Ejecutar un POST desde Swagger UI.

Acción. En Swagger UI utiliza Try it out para enviar POST de orden y observar 201 y Location.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Construcción y evidencia del paso. Desde Swagger UI envía un POST con los datos del ejemplo y comprueba 201 y la cabecera `Location`. La interfaz no cambia el comportamiento del endpoint, solamente lo facilita.

Por qué funciona así. Un ejemplo JSON ilustra el contrato de entrada. Swagger UI permite usarlo para enviar una petición real, pero no crea datos hasta ejecutar el endpoint.

Verificación concreta. Se crea un recurso de verdad desde el explorador HTTP.

Si algo falla. Si no aparece example, comprueba SchemaFilter y el tipo CrearOrdenRequest usado en el esquema.

PASO 2.4.19 — Consultar la orden creada desde la propia UI

Resultado de aprendizaje. Consultar la orden creada desde la propia UI.

Acción. Copia el GUID obtenido y ejecuta GET de detalle desde la UI.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Construcción y evidencia del paso. Utiliza el `Guid` de la orden creada para ejecutar GET por id desde Swagger UI. Debes obtener 200 con los mismos datos que devolvió POST.

Por qué funciona así. Un ejemplo JSON ilustra el contrato de entrada. Swagger UI permite usarlo para enviar una petición real, pero no crea datos hasta ejecutar el endpoint.

Verificación concreta. GET responde 200 con el DTO de la misma orden.

Si algo falla. Si no aparece example, comprueba SchemaFilter y el tipo CrearOrdenRequest usado en el esquema.

PASO 2.4.20 — Crear tests HTTP del documento OpenAPI real

Resultado de aprendizaje. Crear tests HTTP del documento OpenAPI real.

Acción. En ContratoOpenApiTests, usa WebApplicationFactory para solicitar el JSON real.

Archivo de trabajo: **`tests/AceriaApp.Api.Tests/ContratoOpenApiTests.cs`**.

```
using System.Net;
using System.Text.Json;
using Microsoft.AspNetCore.Hosting;
using Xunit;

namespace AceriaApp.Api.Tests;

public class ContratoOpenApiTests
{
    [Fact]
    public async Task OpenApi_ContieneRutasYVersion()
    {
        using var f = new ApiFactory();
        var r = await f.CreateClient().GetAsync("/swagger/v1/swagger.json");
        Assert.Equal(HttpStatusCode.OK, r.StatusCode);
        using var json = JsonDocument.Parse(await r.Content.ReadAsStringAsync());
        Assert.Equal("v1", json.RootElement.GetProperty("info")
            .GetProperty("version").GetString());
        var paths = json.RootElement.GetProperty("paths");
        Assert.True(paths.TryGetProperty("/api/v1/ordenes", out var resource));
        Assert.True(resource.TryGetProperty("get", out _));
        Assert.True(resource.TryGetProperty("post", out _));
    }
}
```

Secuencia de construcción. Crea la clase de tests y sus `using` (`System.Net`, `System.Text.Json`, `Xunit`). Los siguientes pasos amplían esa misma clase: no pegues tres declaraciones diferentes de `ContratoOpenApiTests`.

Por qué funciona así. Descargar el JSON OpenAPI con WebApplicationFactory y comprobar info, paths y schemas es una prueba contractual real; en Production debe respetarse el 404.

Verificación concreta. El test descarga la respuesta del servidor y la analiza.

Si algo falla. Si el JSON no incluye rutas, comprueba MapGet/MapPost y que las pruebas apuntan al servidor correcto.

PASO 2.4.21 — Comprobar rutas GET y POST en paths

Resultado de aprendizaje. Comprobar rutas GET y POST en paths.

Acción. Comprueba info.version y rutas de paths con operaciones get y post.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Construcción y evidencia del paso. En el test `OpenApi_ContieneRutasYVersion` de 2.4.20 comprueba `info.version=v1`, `paths[/api/v1/ordenes].get` y `post`. No crees otro proyecto ni otro documento YAML.

Por qué funciona así. Descargar el JSON OpenAPI con WebApplicationFactory y comprobar info, paths y schemas es una prueba contractual real; en Production debe respetarse el 404.

Verificación concreta. Un cambio de ruta erróneo hace fallar el contrato.

Si algo falla. Si el JSON no incluye rutas, comprueba MapGet/MapPost y que las pruebas apuntan al servidor correcto.

PASO 2.4.22 — Comprobar schema y ejemplo de petición

Resultado de aprendizaje. Comprobar schema y ejemplo de petición.

Acción. Agrega un test que compruebe CrearOrdenRequest y su propiedad example en components.schemas.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Construcción y evidencia del paso. Añade `OpenApi_IncluyeEjemplo` a la clase de tests existente y verifica el `example` dentro del esquema `CrearOrdenRequest`. Ejecuta pruebas para confirmar que Swagger se genera desde la aplicación.

Código que se incorpora en este paso:

```
[Fact]
public async Task OpenApi_IncluyeEjemplo()
{
    using var f = new ApiFactory();
    var r = await f.CreateClient().GetAsync("/swagger/v1/swagger.json");
    using var json = JsonDocument.Parse(await r.Content.ReadAsStringAsync());
    var schemas = json.RootElement
        .GetProperty("components").GetProperty("schemas");
    Assert.True(schemas.TryGetProperty("CrearOrdenRequest", out var schema));
    Assert.True(schema.TryGetProperty("example", out _));
}
```

Por qué funciona así. Descargar el JSON OpenAPI con WebApplicationFactory y comprobar info, paths y schemas es una prueba contractual real; en Production debe respetarse el 404.

Verificación concreta. El esquema se genera a partir del contrato público.

Si algo falla. Si el JSON no incluye rutas, comprueba MapGet/MapPost y que las pruebas apuntan al servidor correcto.

PASO 2.4.23 — Comprobar que Production responde 404 para Swagger

Resultado de aprendizaje. Comprobar que Production responde 404 para Swagger.

Acción. Ejecuta el test en Environment Production y solicita /swagger/v1/swagger.json.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Construcción y evidencia del paso. Añade `Produccion_NoExponeSwagger` usando `WithWebHostBuilder(b => b.UseEnvironment("Production"))`. La petición a `/swagger/v1/swagger.json` debe devolver 404, mientras el documento está disponible en Development.

Código que se incorpora en este paso:

```
[Fact]
public async Task Produccion_NoExponeSwagger()
{
    using var f = new ApiFactory();
    using var other = f.WithWebHostBuilder(
        b => b.UseEnvironment("Production"));
    var r = await other.CreateClient()
        .GetAsync("/swagger/v1/swagger.json");
    Assert.Equal(HttpStatusCode.NotFound, r.StatusCode);
}
```

Por qué funciona así. Descargar el JSON OpenAPI con WebApplicationFactory y comprobar info, paths y schemas es una prueba contractual real; en Production debe respetarse el 404.

Verificación concreta. El endpoint no se publica y responde 404.

Si algo falla. Si el JSON no incluye rutas, comprueba MapGet/MapPost y que las pruebas apuntan al servidor correcto.

PASO 2.4.24 — Reto: documentar PATCH y verificar su summary en JSON

Resultado de aprendizaje. Reto: documentar PATCH y verificar su summary en JSON.

Acción. Reto: añade un resumen específico a PATCH y comprueba que paths muestra summary actualizado.

Referencia para comprobar el resultado: los archivos del proyecto `PROYECTO/2.4-documentacion-openapi` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Construcción y evidencia del paso. Reto opcional: cambia el `WithSummary` del endpoint PATCH y verifica que la propiedad `summary` del documento JSON adopta el nuevo texto. No edites el JSON como si fuera fuente.

Por qué funciona así. Cambiar un WithSummary y ver reflejada la modificación en el JSON demuestra la vinculación de la documentación con el código ejecutado.

Verificación concreta. Se verifica que el documento evoluciona al modificar el código.

Si algo falla. Si el summary no cambia, modifica WithSummary en el endpoint real y vuelve a solicitar el documento.

Caso práctico completo — Contrato de ejemplo y prueba en Production

La sección `components.schemas` documenta objetos que pueden reutilizarse desde varias operaciones. Busca `CrearOrdenRequest`. Debe exponer las propiedades cliente, material y cantidad. El `SchemaFilter` `EjemploCrearOrden` incorpora valores orientativos para esos campos; no crea ninguna orden. El propósito es ayudar a quien consume el API a preparar un cuerpo JSON compatible.

Desde Swagger UI abre el POST de órdenes y utiliza Try it out. Envía un JSON con ACX-001, CHAPA-3MM y cantidad 10; el servidor debe devolver 201 y Location. Guarda el identificador y solicita GET de detalle desde la misma interfaz. Si obtienes una respuesta distinta, revisa el cuerpo, las reglas de validación y la conexión a la base de datos; no concluyas que el contrato está bien porque Swagger haya dibujado el formulario.

Para probar la documentación automáticamente, abre `ContratoOpenApiTests.cs`. La prueba arranca WebApplicationFactory, solicita el JSON real y analiza sus propiedades con JsonDocument. Comprueba `info.version`, las rutas de `paths` y el esquema de CrearOrdenRequest con su `example`. La prueba debe fallar si el endpoint POST se elimina sin actualizar el contrato de la API.

Finalmente crea la fábrica de tests con `UseEnvironment("Production")` y vuelve a solicitar `/swagger/v1/swagger.json`. El método `UseDocumentacion` restringe la publicación a Development; por eso la

respuesta esperada es 404. Esta comprobación enseña a separar la presencia de un paquete de su exposición pública.

Reto final: modifica `WithSummary` de PATCH, arranca de nuevo y comprueba en el JSON de `paths` que el resumen se actualizó. No escribas un documento OpenAPI manual: la prueba debe demostrar que la documentación sigue derivándose de la implementación.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`GET /swagger/v1/swagger.json`	200 en Development
`info.version`	v1
`paths contiene GET y POST`	Rutas descubiertas
`schemas contiene CrearOrdenRequest`	Contrato registrado
`GET /swagger/v1/swagger.json`	404 en Production

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.
